Sprawozdanie z projektu: rozpoznawanie znaków drogowych i odporność modeli na zakłócenia obrazu

Sztuczna inteligencja w systemach informatycznych · 5 listopada 2026

1. Wprowadzenie i cel projektu

Temat: Rozpoznawanie znaków drogowych z wykorzystaniem konwolucyjnych sieci neuronowych oraz analiza odporności modeli na zakłócenia obrazu.

Rozpoznawanie znaków drogowych to zadanie, z którym radzi sobie dziś praktycznie każdy nowszy samochód z systemami wspomagania kierowcy. Kamera w aucie rzadko dostaje jednak ładne, ostre zdjęcie w pełnym słońcu. Obraz bywa rozmyty przez ruch, zaszumiony o zmierzchu albo po prostu ciemny, a sam znak potrafi być częściowo zasłonięty przez gałąź czy naklejkę.

Na zbiorze GTSRB, z którego korzystałem, dobre sieci konwolucyjne dochodzą do około 99% skuteczności. Kolejna próba wyciśnięcia z niego ułamka procenta nie wydała mi się więc szczególnie ciekawa. Zamiast tego postanowiłem sprawdzić, co dzieje się z tą skutecznością, kiedy warunki przestają być idealne, i skupić się na odporności modeli.

Pytanie badawcze: czy uczenie transferowe i odpowiednio dobrana augmentacja danych ograniczają spadek skuteczności klasyfikatora znaków drogowych na zakłóconych zdjęciach?

Żeby odpowiedzieć na nie liczbami, a nie wrażeniem, wytrenowałem kilka wariantów sieci i porównałem ich wyniki na czystych zdjęciach oraz na zdjęciach, które celowo i w kontrolowany sposób zepsułem. Z tego porównania wyszło, które warianty tracą najmniej i co je od pozostałych odróżnia.

Projekt realizowałem w trzech kamieniach milowych:

1. **KM1:** zdefiniowanie problemu, wybór i analiza danych, przygotowanie danych oraz zaplanowanie eksperymentów (rozdziały 2 do 5),
2. **KM2:** implementacja, trening i porównanie wariantów na zbiorze walidacyjnym (rozdziały 6 do 11),
3. **KM3:** jednorazowa ocena finalnego modelu na zbiorze testowym, warstwa wyjaśnialności (XAI) i aplikacja (rozdziały 13 do 17).

To sprawozdanie łączy wszystkie trzy etapy w jedną całość. Po ocenie końcowej dodałem jeszcze eksperyment E5, w którym porównałem strategie fine-tuningu (rozdział 12).

Najważniejsze wyniki w skrócie:

- **finalny model to ResNet18 z wagami ImageNet i augmentacją odpornościową A2.** Na zbiorze testowym ma 99,13% accuracy, 98,44% macro-F1 i 92,72% mCA, czyli średniej accuracy na zakłóconych zdjęciach. Baseline osiąga 94,64% macro-F1 i 76,30% mCA,
- **hipotezy H1 i H2 się potwierdziły,** H3 odrzuciłem, a H4 raczej się nie potwierdziła,
- **o odporności bardziej decyduje augmentacja niż architektura** czy uczenie transferowe,
- **sama zamiana ostatniej warstwy sieci pretrenowanej nie wystarcza.** Trzeba dostroić również wcześniejsze warstwy, najlepiej z mniejszym krokiem uczenia.

2. Zbiór danych

Wybrałem zbiór **GTSRB** (German Traffic Sign Recognition Benchmark), przygotowany przez zespół z Ruhr-Universität Bochum na konkurs IJCNN 2011 [1]. Zdjęcia pochodzą z nagrań kamery samochodowej z niemieckich dróg, a każde z nich to pojedynczy znak wycięty z klatki filmu razem z niewielkim marginesem tła. Dane pobrałem z serwisu Kaggle [2], gdzie są dostępne jako pliki PNG z opisem w plikach CSV.

Element	Zawartość
`Train/`	39 209 zdjęć treningowych w 43 podfolderach (jeden na klasę)
`Test/`	12 630 zdjęć testowych
`Train.csv`, `Test.csv`	rozmiar zdjęcia, współrzędne ramki ze znakiem (ROI), numer klasy, ścieżka do pliku
`Meta/`	wzorcowe piktogramy wszystkich 43 klas

Dlaczego właśnie ten zbiór

Przy wyborze zależało mi na kilku rzeczach:

- **Sprawdzone źródło.** GTSRB to uznany benchmark opisany w recenzowanej publikacji, z ręcznie weryfikowanymi etykietami, więc nie musiałem się zastanawiać, czy dane są wiarygodne.
- **Rozsądna wielkość.** Ponad 51 tysięcy zdjęć to wystarczająco dużo, żeby porównanie architektur miało sens, a jednocześnie na tyle mało, że wszystkie treningi zmieściły się w kilku godzinach na laptopie, a nie w kilku nocach.
- **Oficjalny zbiór testowy** pochodzący z innych nagrań niż treningowy, czyli uczciwy egzamin na koniec.
- **Naturalnie trudne zdjęcia.** Część zdjęć jest ciemna, mała albo nieostra już w oryginale, więc temat odporności nie jest wymyślony na siłę.
- **Ramki ROI,** czyli dokładne położenie znaku na każdym zdjęciu. Przydały się w analizie wyjaśnialności (rozdział 15), kiedy sprawdzałem, czy model faktycznie patrzy na znak, czy na tło.

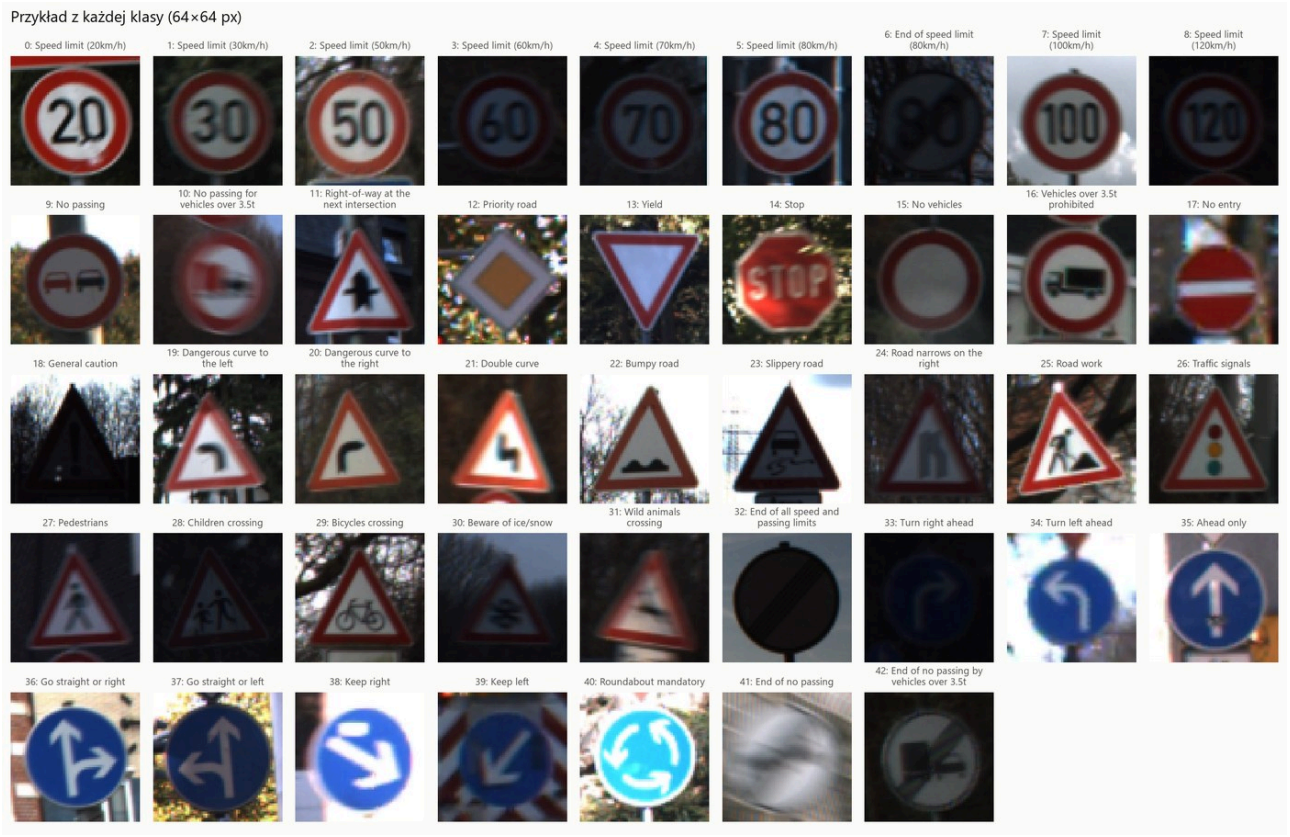
Zbiór testowy od samego początku odłożyłem na bok. Na etapie analizy danych korzystałem tylko z etykiet i rozmiarów zapisanych w `Test.csv`, żeby porównać rozkłady klas. Samych zdjęć testowych nie otwierałem aż do oceny końcowej (rozdział 13), w której użyłem ich jeden raz.

3. Analiza eksploracyjna danych

Analizę zrobiłem w notebooku `notebooks/01_eda.ipynb`, a wykresy i policzone statystyki zapisałem w `results/eda/`. Przy każdej obserwacji od razu opisuję, jaką decyzję na jej podstawie podjąłem, bo właśnie po to tę analizę robiłem.

3.1. Przegląd klas

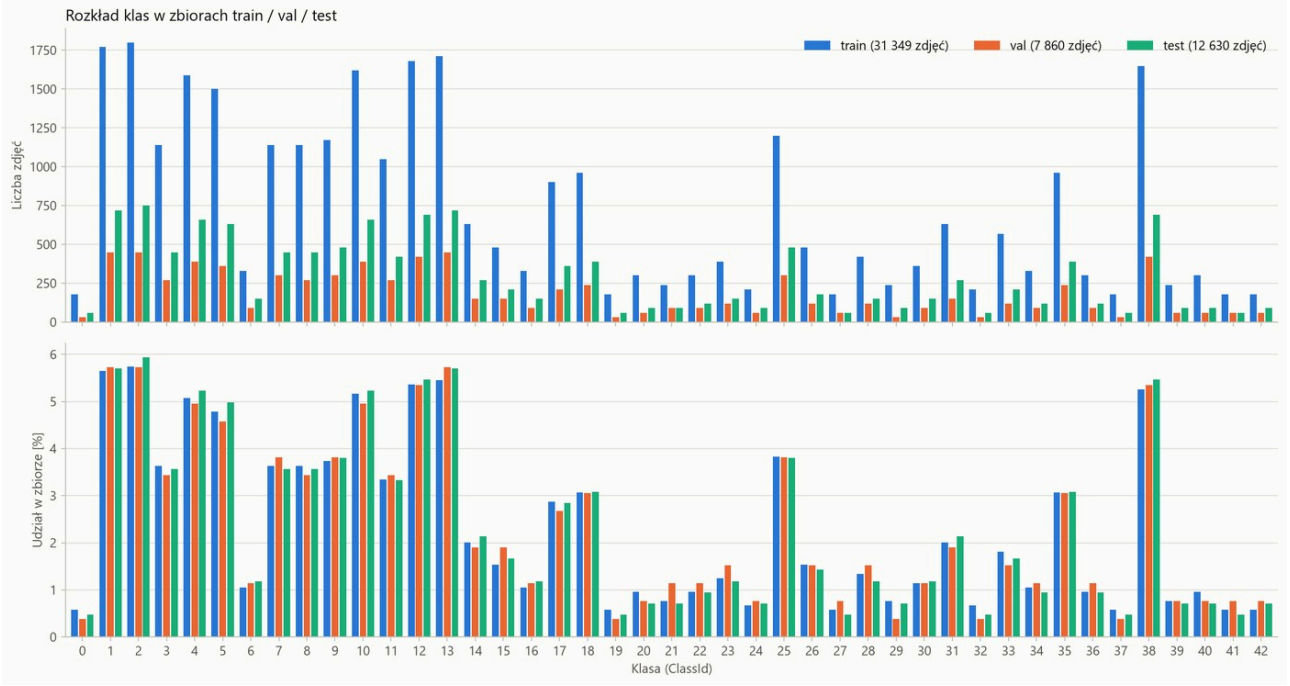
W zbiorze są 43 klasy: ograniczenia prędkości, zakazy, nakazy, ostrzeżenia i znaki informacyjne. Już na pierwszy rzut oka widać, że niektóre są do siebie łudząco podobne. Najbardziej rzucają się w oczy ograniczenia prędkości (20, 30, 50, 60, 70, 80, 100 i 120 km/h), które różnią się wyłącznie cyframi w środku tego samego czerwonego okręgu. Obstawiałem, że to właśnie tam model będzie mylił się najczęściej. Jak się okazało w analizie błędów (rozdział 14), myliłem się.



Rys. 1. Przykładowe zdjęcie z każdej z 43 klas.

3.2. Liczność klas

Zbiór jest mocno niezbalansowany. Najrzadsza klasa (0, ograniczenie do 20 km/h) ma 210 zdjęć treningowych, a najliczniejsza (2, ograniczenie do 50 km/h) aż 2 250, czyli prawie jedenaście razy więcej. W zbiorze testowym proporcje wyglądają niemal identycznie.

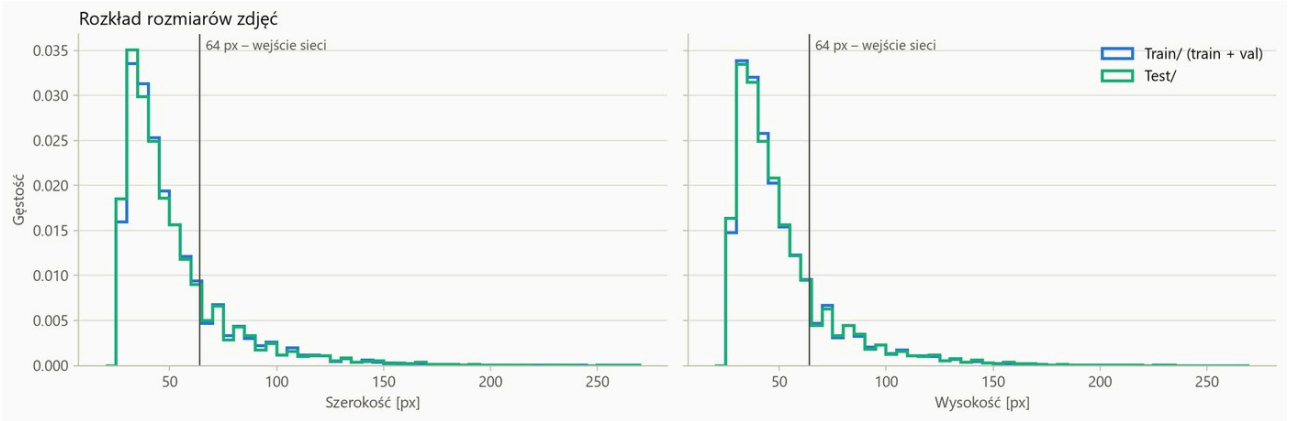


Rys. 2. Liczba zdjęć (góra) oraz udział procentowy (dół) każdej klasy w zbiorze treningowym, walidacyjnym i testowym.

Decyzja: samo accuracy byłoby tu złudne, bo model mógłby mieć świetny wynik ogólny, a jednocześnie regularnie mylić rzadkie znaki. Dlatego jako główną metrykę porównawczą przyjąłem **macro-F1**, w której każda klasa liczy się tak samo, niezależnie od liczby zdjęć.

3.3. Rozmiary zdjęć

Zdjęcia są małe: od 25 do około 250 pikseli szerokości, z medianą 43 px. Prawie 80% z nich ma oba boki krótsze niż 64 px. Proporcje są praktycznie kwadratowe (mediana stosunku szerokości do wysokości wynosi dokładnie 1,0).

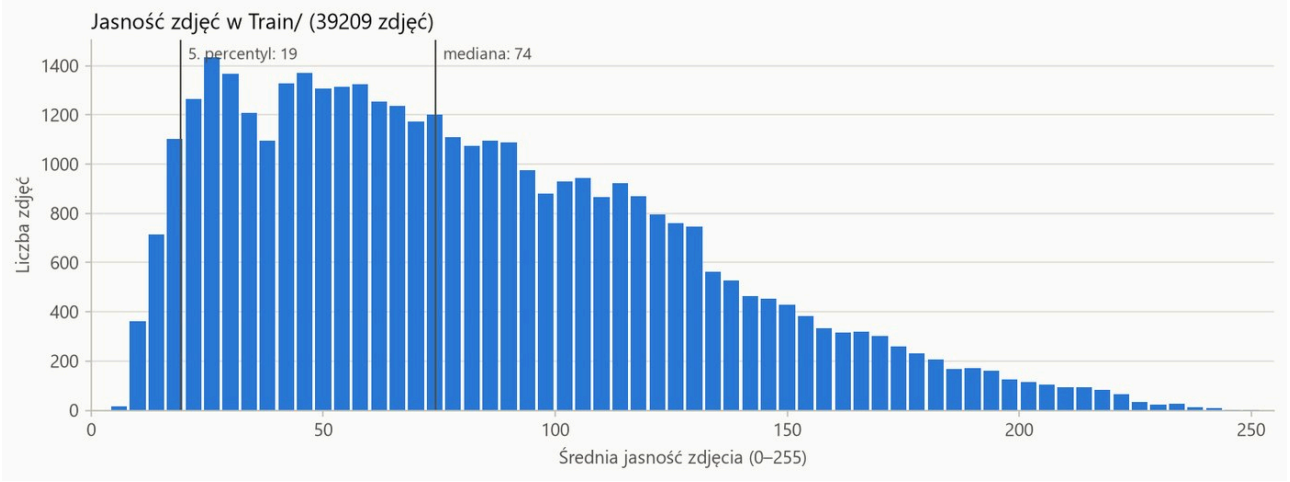


Rys. 3. Rozkład szerokości i wysokości zdjęć.

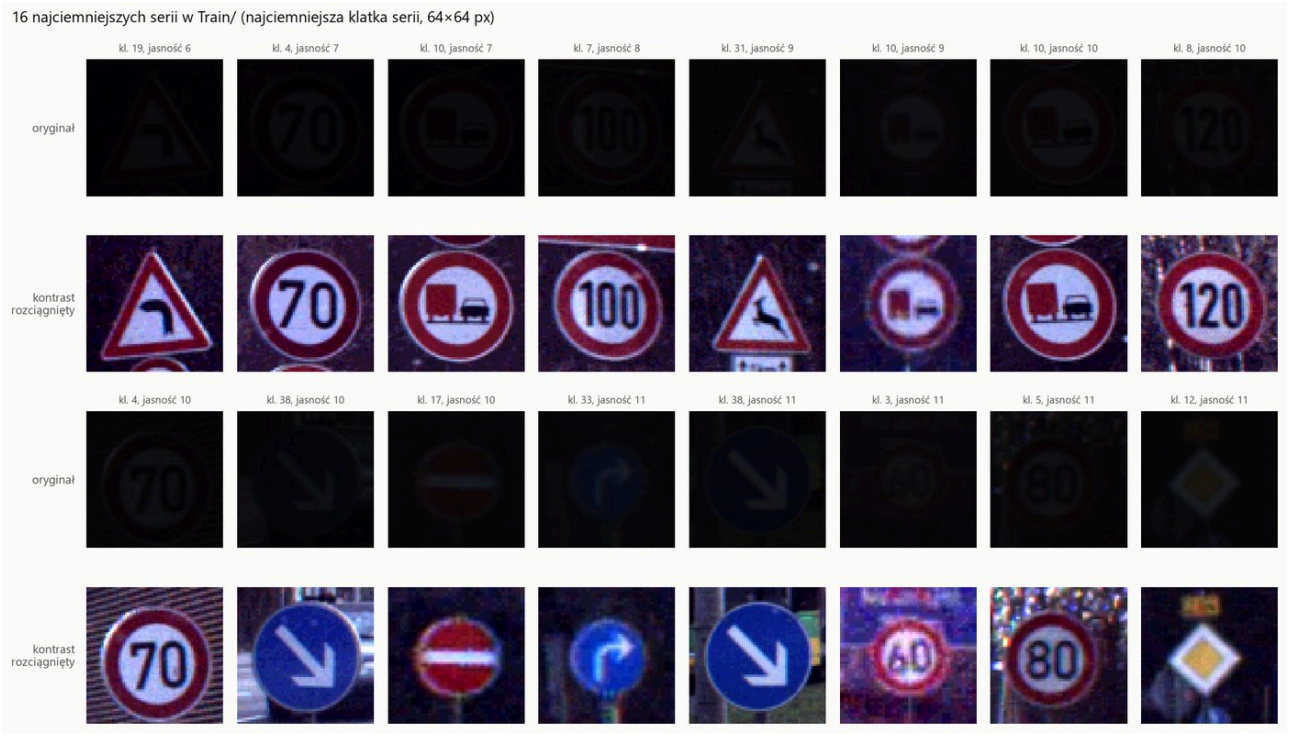
Decyzja: wszystkie zdjęcia skaluję do **64×64 px**. Dla większości danych to lekkie powiększenie, więc nic nie tracę, a trening zostaje szybki. Skoro zdjęcia są kwadratowe, przeskalowanie nie rozciąga znaków.

3.4. Jasność

Ta obserwacja trochę mnie zaskoczyła. Mediana średniej jasności zdjęcia to tylko 74 w skali od 0 do 255, około 14% zdjęć ma średnią jasność poniżej 30, a 30% poniżej 50. Najciemniejsze przykłady są trudne do rozpoznania nawet dla człowieka.



Rys. 4. Rozkład średniej jasności zdjęć treningowych.

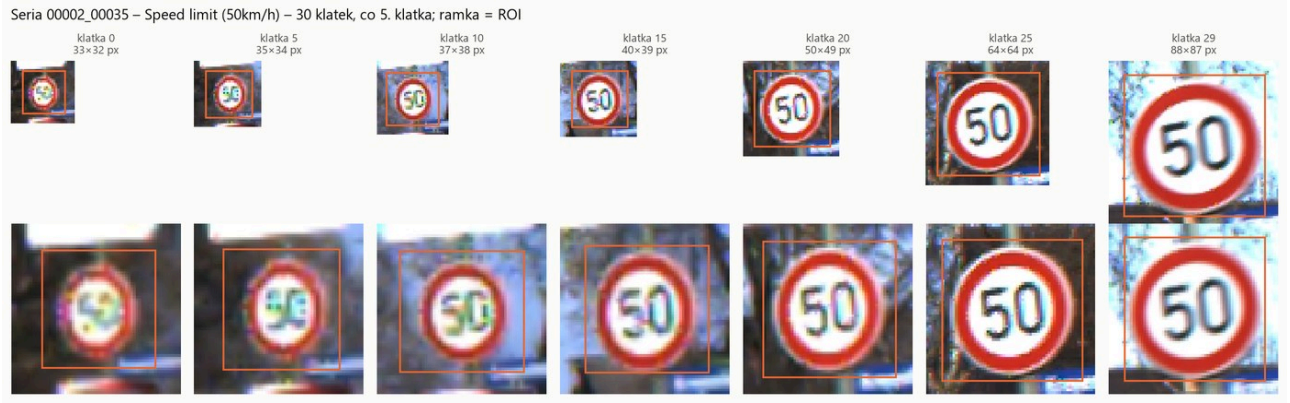


Rys. 5. Przykłady najciemniejszych zdjęć w zbiorze treningowym.

Decyzja: ciemne zdjęcia nie są w GTSRB wyjątkiem, tylko codziennością, i musiałem to uwzględnić w dwóch miejscach. Augmentacja przyciemniająca nie może być zbyt agresywna, bo nałożona na i tak ciemne zdjęcie potrafi całkiem wymazać znak (rozdział 4.3). Z kolei zakłócenie „przyciemnienie", którym sprawdzałem odporność, musi być na tyle mocne, żeby wychodzić poza to, co model i tak regularnie widzi w danych (rozdział 5.4).

3.5. Serie zdjęć tego samego znaku

To była dla mnie najważniejsza obserwacja całej analizy. Zbiór treningowy nie składa się z 39 tysięcy niezależnych zdjęć, tylko z **1 307 serii** (tracków). Każda seria to około 30 kolejnych klatek nagrania tego samego fizycznego znaku, który rośnie w miarę zbliżania się samochodu. Numer serii da się odczytać z nazwy pliku (`<klasa>_<seria>_<klatka>.png`), a na jedną klasę przypada od 7 do 75 serii.



Rys. 6. Co piąta klatka jednej serii (ograniczenie do 50 km/h). Górny rząd pokazuje oryginalne rozmiary, dolny po przeskalowaniu do 64×64 px. Pomarańczowa ramka to ROI.

Decyzja: sąsiednie klatki są praktycznie identyczne. Gdybym wydzielił walidację losowo po pojedynczych zdjęciach, prawie te same obrazy trafiłyby jednocześnie do treningu i do walidacji, a model „zdawałby egzamin", znając odpowiedzi. Dlatego dane dzielę **po seriach** (rozdział 4.1).

3.6. Ramki ROI

W plikach CSV każde zdjęcie ma zapisane współrzędne ramki wokół znaku. Zwykle ramka jest odsunięta od krawędzi o około 5 px z każdej strony. Sprawdziłem, że po przeskalowaniu do 64×64 px ramki nadal dobrze obejmują znak (widać to na rys. 6). Wykorzystałem je w rozdziale 15, żeby policzyć, jaka część uwagi modelu z map Grad-CAM przypada na sam znak, a jaka na tło.

4. Przygotowanie danych

Całe przygotowanie danych zamknąłem w jednym module, `src/data.py`. Dzięki temu każdy eksperyment korzystał z dokładnie tego samego podziału, przetwarzania i augmentacji, więc różnice w wynikach wynikają z modeli, a nie z danych.

4.1. Podział na zbiór treningowy i walidacyjny

Walidację wydzieliłem z oficjalnego zbioru treningowego, zgodnie z wnioskiem z rozdziału 3.5 po seriach: każda seria trafia w całości albo do treningu, albo do walidacji. Użyłem do tego `StratifiedGroupKFold` z biblioteki scikit-learn (5 foldów, grupy to serie, stratyfikacja po klasach, ziarno losowości 42), a jeden fold, czyli około 20% serii, przeznaczyłem na walidację. Stratyfikacja pilnuje, żeby proporcje klas w obu zbiorach były takie same.

Zbiór	Zdjęcia	Serie	Przeznaczenie
Treningowy	31 349	1 045	uczenie modeli
Walidacyjny	7 860	262	wybór wariantu, early stopping, ocena odporności
Testowy	12 630	brak danych o seriach	jednorazowa ocena finalnego modelu

Po wykonaniu podziału sprawdziłem, czy wszystko się zgadza:

- trening i walidacja nie mają ani jednej wspólnej serii,
- w walidacji są wszystkie 43 klasy, każda z co najmniej 30 zdjęciami,
- udział żadnej klasy nie różni się między treningiem a walidacją o więcej niż 0,38 punktu procentowego,
- podział jest powtarzalny: listę serii walidacyjnych zapisałem w pliku `splits/val_tracks.json`, który jest w repozytorium i z którego korzystają wszystkie skrypty.

Ograniczenie: klasy 0, 19, 29, 32 i 37 mają w walidacji tylko po jednej serii, bo w całym zbiorze mają ich bardzo mało. Ich wynik walidacyjny opiera się więc na jednym fizycznym znaku i traktowałem go z przymrużeniem oka. Rzetelną ocenę tych klas dał dopiero zbiór testowy.

4.2. Przetwarzanie wstępne

Każde zdjęcie przechodzi te same kroki: konwersję do RGB, przeskalowanie do 64×64 px z antyaliasingiem, zamianę na tensor z wartościami od 0 do 1 oraz normalizację średnią i odchyleniem standardowym ze zbioru ImageNet. Ramki ROI skaluję razem ze zdjęciem.

Normalizację ImageNet stosuję dla wszystkich modeli, także dla własnej sieci uczonej od zera, choć ta formalnie jej nie potrzebuje. Chodzi o to, żeby warianty różniły się wyłącznie architekturą i sposobem inicjalizacji wag, a nie tym, jak przygotowano dla nich dane.

Przy okazji trafiłem na problem z wydajnością: na Windowsie samo wczytywanie plików PNG zajmowało około 70 s na epokę. Dodałem więc opcję jednorazowego wczytania i przeskalowania

wszystkich zdjęć do pamięci RAM. Wyniki są identyczne co do bitu, a epoka trwa około 13 s. Przy kilkunastu treningach to robi sporą różnicę.

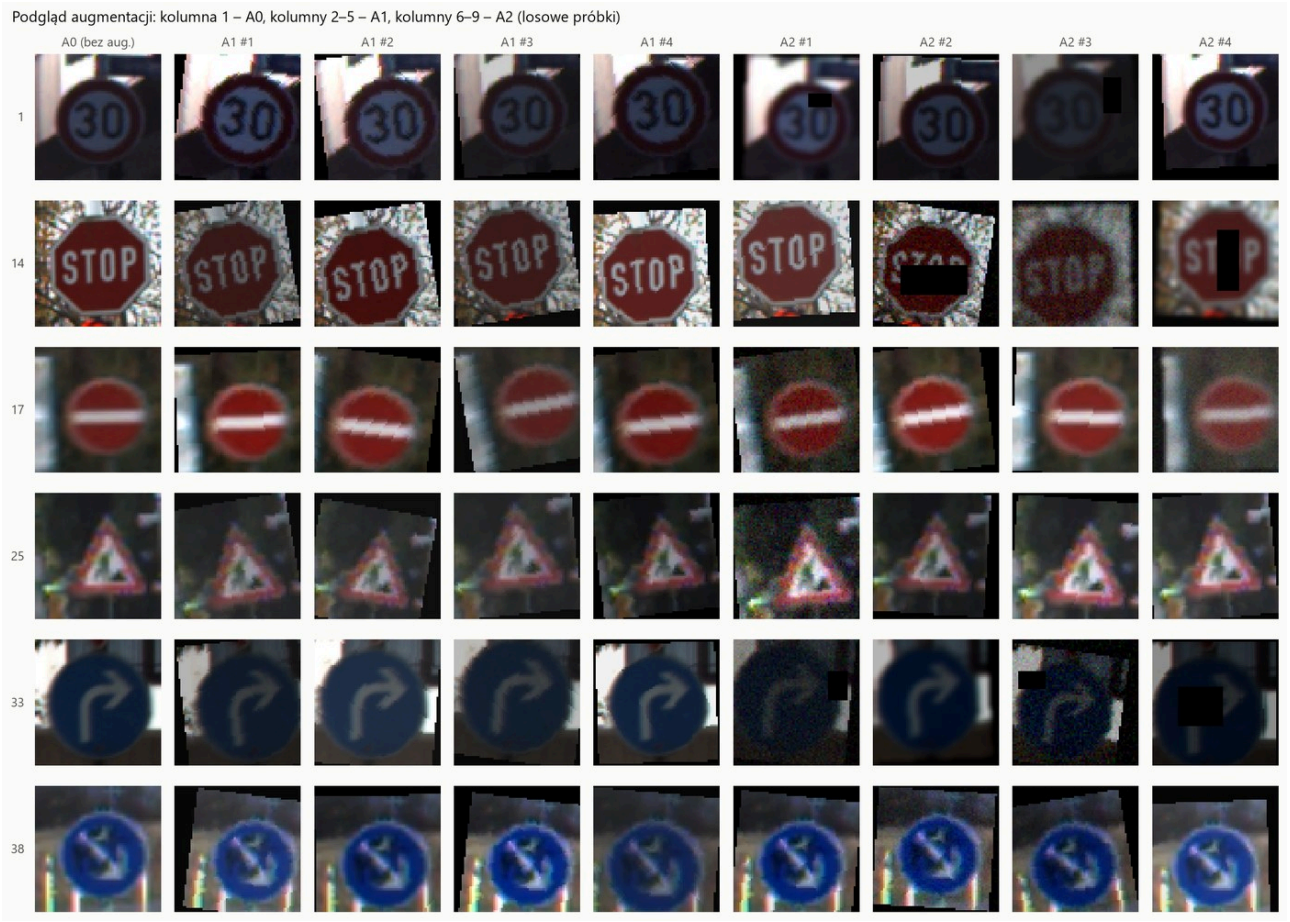
4.3. Augmentacja danych

Przygotowałem trzy poziomy augmentacji, które porównałem w eksperymentach:

Poziom	Operacje
A0 brak	tylko przetwarzanie wstępne (ten sam zestaw służy do walidacji i testu)
A1 geometryczna	A0 + losowy obrót ±10°, przesunięcie do 10%, skalowanie 0,9 do 1,1, zmiana jasności i kontrastu ±30%
A2 odpornościowa	A1 + z prawdopodobieństwem 0,3 każda: rozmycie Gaussa (σ 0,1 do 1,5), przyciemnienie (×0,5 do 0,9), szum Gaussa (σ 0,01 do 0,05), wymazanie prostokątnego fragmentu (2% do 15% powierzchni)

Dwie rzeczy wymagają tu wyjaśnienia:

- **Celowo nie używam odbić lustrzanych,** choć to jedna z najpopularniejszych augmentacji. Przy znakach drogowych odbicie zmienia ich znaczenie: odbity nakaz skrętu w prawo staje się nakazem skrętu w lewo, a model dostałby zdjęcie z błędną etykietą.
- **A2 osłabiłem po obejrzeniu danych.** W pierwszej wersji przyciemniała zdjęcia nawet do 30% jasności i dodawała szum o σ do 0,08. Na podglądzie okazało się, że na zdjęciach, które i tak były ciemne, znak potrafił całkowicie zniknąć, więc model uczyłby się na obrazach, których nie da się rozpoznać. Zmniejszyłem zakresy do wartości z tabeli. Zrobiłem to, zanim wytrenowałem jakikolwiek model, wyłącznie na podstawie analizy danych, żeby nie było podejrzeń, że dopasowuję augmentację do wyników.



Rys. 7. Podgląd augmentacji po korekcie. Kolumna 1 to A0, kolumny 2 do 5 to losowe próbki A1, kolumny 6 do 9 to losowe próbki A2.

W żadnej augmentacji nie ma kompresji JPEG i jest to zamierzone. JPEG posłużył jako zakłócenie, którego model nie zobaczył w treningu ani razu (hipoteza H4).

5. Projekt eksperymentu

Cały plan eksperymentów ustaliłem przed pierwszym treningiem. Każdy eksperyment odpowiada na konkretną hipotezę, a sposób wyboru najlepszego modelu zapisałem z góry. Dzięki temu nie testowałem na ślepo kolejnych modeli, tylko sprawdzałem założenia z tego rozdziału.

5.1. Hipotezy

1. **H1 (uczenie transferowe):** modele startujące z wag wytrenowanych na ImageNet osiągną wyższe macro-F1 i szybciej się nauczą niż te same architektury uczone od zera.
2. **H2 (augmentacja):** augmentacja odpornościowa A2 zmniejszy spadek skuteczności na zakłóconych zdjęciach wyraźnie bardziej niż geometryczna A1, a na czystych danych straci najwyżej 1 punkt procentowy macro-F1.
3. **H3 (rozmiar modelu):** MobileNetV3-Small osiągnie wynik zbliżony do ResNet18 (różnica macro-F1 poniżej 1 punktu procentowego), mając przy tym kilka razy mniej parametrów i krótszy czas predykcji.
4. **H4 (uogólnienie odporności):** odporność nabyta dzięki augmentacji przeniesie się przynajmniej częściowo na zakłócenie, którego model nie widział w treningu (kompresja JPEG).

5.2. Baseline

Jako punkt odniesienia przyjąłem **własną, prostą sieć konwolucyjną (SimpleCNN)** uczoną od zera i bez augmentacji. Ma trzy bloki (konwolucja, normalizacja wsadowa, ReLU, max pooling) i klasyfikator z warstwą w pełni połączoną. Bez takiego punktu odniesienia trudno byłoby ocenić, czy wynik bardziej złożonych modeli to faktycznie sukces, czy coś, co da się osiągnąć najprostszym podejściem.

5.3. Warianty rozwiązania

Warianty różnią się w trzech wymiarach:

- **architektura:** SimpleCNN, ResNet18 [3] i MobileNetV3-Small [4],
- **inicjalizacja wag:** od zera albo wagi pretrenowane na ImageNet (dla ResNet18 i MobileNetV3),
- **augmentacja:** A0, A1 lub A2 (rozdział 4.3).

ResNet18 wybrałem jako sprawdzoną sieć średniej wielkości, a MobileNetV3-Small jako lekką sieć projektowaną z myślą o urządzeniach mobilnych, co dobrze pasuje do zastosowania w samochodzie. Wszystkie warianty trenowałem w identycznych warunkach: wejście 64×64 px, optymalizator AdamW, najwyżej 20 epok, early stopping na macro-F1 walidacji z cierpliwością 5 epok i ziarno losowości 42.

5.4. Zakłócenia do oceny odporności

Odporność mierzyłem na zbiorze walidacyjnym z nałożonymi zakłóceniami, wzorując się na metodologii z badań nad odpornością sieci na typowe zniekształcenia obrazu [5]. Wybrałem pięć zakłóceń, z których każde odpowiada czemuś, co realnie może spotkać kamerę w samochodzie, i każde występuje w trzech poziomach nasilenia. Zakłócenia są deterministyczne (stałe ziarno), więc każdy model dostaje dokładnie te same zepsute obrazy.

Zakłócenie	Odpowiednik w rzeczywistości	Poziom 1	Poziom 2	Poziom 3	Obecne w A2
Rozmycie Gaussa	ruch pojazdu, brak ostrości	σ = 0,8	σ = 2,0	σ = 3,0	tak (σ do 1,5)
Szum Gaussa	słabe oświetlenie, tania kamera	σ = 0,03	σ = 0,06	σ = 0,10	tak (σ do 0,05)
Przyciemnienie	zmierzch, noc	×0,6	×0,4	×0,25	tak (do ×0,5)
Zasłonięcie	gałąź, naklejka, śnieg	10% ramki ROI	20% ramki ROI	30% ramki ROI	tak (wymazanie fragmentu)
Kompresja JPEG	słaba jakość obrazu z kamery	jakość 30	jakość 15	jakość 8	**nie**

Dla rozmycia, szumu i przyciemnienia dobrałem poziomy tak, żeby pierwszy mieścił się w zakresie, który model widzi w treningu z A2, a drugi i trzeci wychodziły poza niego. Dzięki temu sprawdziłem nie tylko, czy model radzi sobie z tym, co już zna, ale też czy jego odporność sięga dalej. Zasłonięcie umieściłem wewnątrz ramki ROI, bo zakrycie kawałka tła niczego by nie testowało. Przed pierwszym treningiem jeszcze raz obejrzałem zakłócone obrazy (rozdział 7.1).

5.5. Metryki

Metryka	Co mierzy	Hipoteza
Accuracy i macro-F1 na czystych danych	ogólną skuteczność, macro-F1 z równą wagą dla rzadkich klas	H1, H3
mCA (mean Corruption Accuracy)	średnią accuracy po 5 zakłóceniach i 3 poziomach, czyli odporność	H2, H4
Względny spadek	(accuracy na czystych danych minus mCA) podzielone przez accuracy na czystych danych	H2
Liczba parametrów i czas predykcji na CPU	koszt modelu (mediana ze 100 predykcji pojedynczego zdjęcia)	H3
Macierz pomyłek i F1 dla każdej klasy	które klasy się mylą	analiza błędów

5.6. Plan eksperymentów

Etap	Cel	Warianty	Treningów	Hipoteza
E1	architektura i uczenie transferowe	SimpleCNN; ResNet18 i MobileNetV3-Small, każdy od zera i pretrenowany; wszystkie z A0	5	H1, H3
E2	wpływ augmentacji	SimpleCNN oraz najlepszy wariant ResNet18 i MobileNetV3 z E1, każdy z A1 i A2	6	H2
E3	ocena odporności	wszystkie 11 modeli z E1 i E2 ocenione na zakłóconej walidacji	0 (tylko ewaluacja)	H2, H4
E4	stabilność wyniku	zwycięski wariant z trzema różnymi ziarnami losowości	3	wiarygodność wyniku

Łącznie dawało to 14 treningów. Po ocenie końcowej dodałem jeszcze etap E5, czyli porównanie strategii fine-tuningu na zwycięskim wariancie (3 treningi, rozdział 12).

5.7. Reguła wyboru najlepszego rozwiązania

Regułę wyboru zapisałem przed treningami, żeby nie kusiło mnie dopasowanie jej do wyników:

1. odrzucam warianty, których macro-F1 na czystej walidacji jest niższe od najlepszego o więcej niż 1 punkt procentowy,
2. spośród pozostałych wybieram wariant z najwyższym mCA,
3. jeśli różnica mCA jest mniejsza niż 0,5 punktu procentowego, wygrywa model mniejszy i szybszy.

Zbiór testowy nie bierze udziału w żadnym z tych kroków. Użyłem go raz, do oceny zwycięskiego modelu i baseline'u (rozdział 13).

6. Implementacja i środowisko

Cały kod jest w repozytorium, a każdy etap da się uruchomić jednym poleceniem. Zależało mi na tym, żeby eksperymenty były w pełni powtarzalne i żeby nic nie trzeba było klikać ręcznie.

Moduł	Co robi
`src/data.py`	wczytywanie danych, podział po seriach, augmentacje A0/A1/A2
`src/models.py`	SimpleCNN, ResNet18 i MobileNetV3-Small, od zera albo z wagami ImageNet
`src/corruptions.py`	pięć zakłóceń w trzech poziomach, deterministycznie
`src/train.py`	trening na podstawie pliku konfiguracyjnego, strategie fine-tuningu, early stopping, zapis historii
`src/evaluate.py`	metryki na czystych i zakłóconych danych, czas predykcji na CPU
`src/run_experiments.py`	uruchamia etapy E1 do E5 i sam wybiera warianty według reguł z rozdziału 5.7
`src/summarize.py`	tabela zbiorcza i wykresy z eksperymentów na walidacji
`src/final_eval.py`	jednorazowa ocena na zbiorze testowym
`src/xai.py`	Grad-CAM, udział uwagi w ramce znaku, galeria błędów, kalibracja i próg pewności
`app/app.py`	aplikacja Gradio

6.1. Modele i adaptacja do małych obrazów

SimpleCNN zbudowałem tak, jak zaplanowałem: trzy bloki po dwie konwolucje 3×3 z normalizacją wsadową i max poolingiem (32, 64 i 128 kanałów), potem global average pooling, dropout 0,3 i warstwa liniowa. Ma 293 tysiące parametrów.

Przy ResNet18 i MobileNetV3-Small musiałem podjąć decyzję, której nie przewidziałem na etapie planowania. Obie sieci są projektowane pod obrazy 224×224 i zmniejszają je po drodze 32 razy. Przy naszych 64×64 px ostatnia mapa cech miałaby więc rozmiar 2×2, co jest mało i dla samej klasyfikacji, i przede wszystkim dla Grad-CAM (rozdział 15), który pokazałby jedną wielką plamę. Dlatego:

- w ResNet18 ustawiłem krok pierwszej konwolucji na 1 i usunąłem max pooling, dzięki czemu ostatnia mapa ma 8×8,
- w MobileNetV3-Small ustawiłem krok pierwszej konwolucji na 1, co daje ostatnią mapę 4×4.

Zmieniłem tylko kroki, a nie kształty wag, więc wagi pretrenowane na ImageNet ładują się bez żadnych zmian. W obu sieciach podmieniłem ostatnią warstwę (głowicę) na nową, z 43 wyjściami.

Model	Parametry	Ostatnia mapa cech
SimpleCNN	293 003	128 × 8 × 8
ResNet18	11 198 571	512 × 8 × 8
MobileNetV3-Small	1 561 931	576 × 4 × 4

6.2. Trening i fine-tuning

Wszystkie warianty z etapów E1 do E4 trenowałem w identycznych warunkach: AdamW (lr 0,001, weight decay 0,0001), harmonogram cosinusowy, batch 128, maksymalnie 20 epok i early stopping na macro-F1 walidacji z cierpliwością 5 epok. Zapisywane są wagi z najlepszej epoki.

W modelach pretrenowanych stosowałem pełny fine-tuning (*full*): po podmianie głowicy uczyłem całą sieć, wszystkie warstwy z tym samym learning rate. Nic nie było zamrożone, więc wagi z ImageNet służyły jako punkt startowy i były dalej dostrajane do znaków drogowych. W etapie E5 (rozdział 12)

porównałem tę strategię z trzema innymi: trenowaniem samej głowicy, niższym learning rate dla pretrenowanej części sieci oraz treningiem dwuetapowym.

Do przyspieszenia użyłem treningu w mieszanej precyzji. Na mojej karcie (RTX 4070 Laptop) najszybszy okazał się format bf16 w połączeniu z układem pamięci channels_last: około 1330 obrazów na sekundę dla ResNet18, podczas gdy fp16 dawało zaskakująco słabe 300. Na wynik nie ma to wpływu, a epoka ResNeta trwa około 28 s.

6.3. Zakłócenia i ewaluacja

Zakłócenia zaimplementowałem z parametrami z rozdziału 5.4. Losowość (szum, położenie zasłonięcia) zależy od ziarna, numeru zdjęcia, typu i poziomu zakłócenia, więc każdy model widzi dokładnie te same zepsute obrazy. Zasłonięcie to czarny prostokąt o proporcjach ramki ROI, zawsze w całości wewnątrz niej.

Po każdym treningu skrypt liczy metryki na czystej walidacji (accuracy, macro-F1, F1 dla każdej klasy, macierz pomyłek) oraz accuracy dla każdego z 15 zakłóceń, a z nich mCA i względny spadek. Czas predykcji mierzę na procesorze dla pojedynczego zdjęcia (mediana ze 100 powtórzeń po rozgrzewce), na 4 wątkach i dodatkowo na 1 wątku. Przy domyślnych 24 wątkach mały MobileNet wychodził sztucznie wolny, bo narzut na koordynację wątków był większy niż samo liczenie.

Zbiór testowy jest w kodzie zablokowany: ewaluacja na nim wymaga jawnej flagi, której użyłem tylko raz, przy ocenie końcowej.

6.4. Automatyzacja eksperymentów

Skrypt `run_experiments.py` uruchamia kolejne etapy i podejmuje decyzje dokładnie według reguł z rozdziału 5.7. Po E1 sam wybiera warianty do E2, a po E3 stosuje regułę wyboru zwycięzcy. Każdy wybór zapisuje do pliku JSON razem z uzasadnieniem i liczbami (`results/e2_selection.json`, `results/e4_selection.json`). Dzięki temu wybór nie zależał od mojego widzimisię, a każdy krok da się sprawdzić. Przerwany przebieg można wznowić, bo gotowe treningi są pomijane.

6.5. Środowisko i reprodukowalność

Element	Wartość
Język i biblioteki	Python 3.12, PyTorch 2.14 z torchvision, scikit-learn, pandas, matplotlib, pytorch-grad-cam, Gradio
Sprzęt	laptop z kartą NVIDIA RTX 4070 Laptop (8 GB VRAM), CUDA 13.2
Repozytorium	github.com/BPajda/GTSRB_robustness

Zadbałem o to, żeby wyniki dało się odtworzyć:

- wszędzie ustawiam to samo ziarno losowości (42): w Pythonie, NumPy, PyTorch i loaderach danych,
- podział na trening i walidację jest zapisany w repozytorium (`splits/val_tracks.json`), więc nie losuje się od nowa przy każdym uruchomieniu,
- każdy trening ma własny plik konfiguracyjny w `configs/`, a jego wyniki (konfiguracja, historia, metryki, wykresy) lądują w `results/<nazwa runu>/`,
- wersje bibliotek są przypięte w `requirements.txt`, a `README.md` opisuje instalację i uruchomienie wszystkich etapów,

- notebook EDA jest zapisany razem z wynikami, więc analizę można przejrzeć bez uruchamiania kodu.

Samego zbioru danych i wag modeli nie umieściłem w repozytorium, bo są za duże. W `README.md` opisałem, skąd pobrać dane i jak odtworzyć wagi.

7. Przebieg eksperymentów

7.1. Kontrola zakłóceń przed treningiem

Zanim wytrenowałem pierwszy model, obejrzałem zakłócone obrazy. Wygenerowałem podgląd dla sześciu zdjęć walidacyjnych, w tym dwóch bardzo ciemnych.



Rys. 8. Pięć zakłóceń w trzech poziomach na sześciu zdjęciach walidacyjnych. Pomarańczowa ramka w pierwszej kolumnie to ROI.

Zakłócenia wyglądają tak, jak chciałem. Poziom 1 jest delikatny i znak jest w pełni czytelny, a poziom 3 jest naprawdę trudny: rozmyte cyfry ograniczeń prędkości są nie do odczytania, a 30% zasłonięcia potrafi zakryć połowę liczby. Na dwóch najciemniejszych zdjęciach przyciemnienie na poziomie 3 daje praktycznie czarny obraz, z którym nie poradziłby sobie nawet człowiek. Uznałem, że to w porządku, bo ten poziom celowo wychodzi poza zakres treningu. Parametrów nie zmieniałem.

7.2. Treningi

Treningi uruchamiałem w kilku sesjach, bo etapy zależą od siebie (E2 korzysta z wyników E1, a E4 z wyników E2), a krótsze sesje łatwiej było mi zmieścić w ciągu dnia.

Sesja	Etapy	Treningów	Czas samego treningu
1	E1	5	46 min
2	E2	6	58 min
3	E3, pomiar czasów, E4	2	18 min
4 (po ocenie końcowej)	E5	3	25 min

Dla E1 do E4 to 13 treningów i około 2 godziny samego treningu, do tego około 2 minuty ewaluacji na każdy model. Na etapie planowania szacowałem całość na około godzinę, ale tamto oszacowanie opierało się na czasie samego wczytywania danych, bez obliczeń sieci. W E4 trenowałem dwa nowe seedy (43 i 44), bo przebieg z seedem 42 już istniał z E2.

7.3. Wiarygodny pomiar czasu predykcji

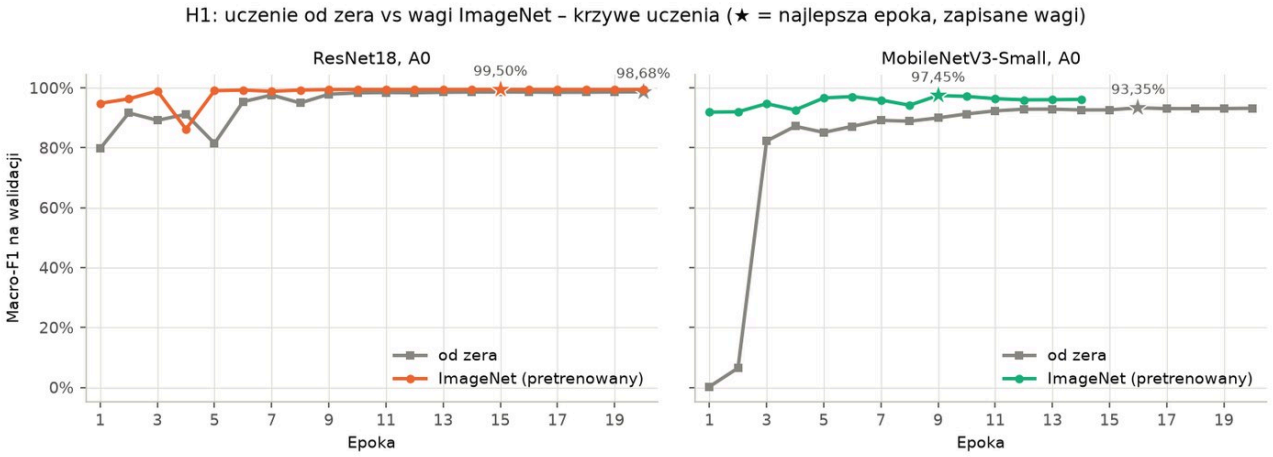
Dokładność modelu nie zależy od tego, co robię na komputerze w tle. Czas predykcji już tak, bo to zwykły pomiar stoperem. Podczas pierwszych dwóch sesji pracowałem obok, więc czasy mierzone zaraz po treningu nie były porównywalne. Testowy pomiar przy włączonych w tle programach dawał dla ResNet18 około 74 ms zamiast 27 ms.

Dlatego dodałem tryb, który mierzy czasy od nowa dla wszystkich gotowych modeli, i uruchomiłem go w sesji 3, na bezczynnym komputerze z zamkniętymi programami, **przed** wyborem zwycięzcy. Wszystkie modele były więc mierzone w jednej sesji i w identycznych warunkach (plan zasilania z maksymalnym stanem procesora 99%, czyli bez dynamicznego zwiększania częstotliwości taktowania procesora). Do sprawozdania i do reguły wyboru trafiły tylko te pomiary.

8. E1: architektura i uczenie transferowe

W E1 wytrenowałem pięć wariantów bez augmentacji (A0): baseline SimpleCNN oraz ResNet18 i MobileNetV3-Small, każdy od zera i z wagami ImageNet. Wszystkie liczby w rozdziałach 8 do 12 dotyczą zbioru walidacyjnego.

Model	Inicjalizacja	macro-F1	Accuracy	mCA	Najlepsza epoka
SimpleCNN (baseline)	od zera	95,25%	97,35%	76,33%	16
ResNet18	od zera	98,68%	99,25%	86,98%	20
ResNet18	ImageNet	**99,50%**	**99,67%**	**87,36%**	15
MobileNetV3-Small	od zera	93,35%	95,64%	75,84%	16
MobileNetV3-Small	ImageNet	97,45%	98,30%	78,02%	9



Rys. 9. Macro-F1 na walidacji po każdej epoce, uczenie od zera i z wagami ImageNet. Gwiazdka oznacza najlepszą epokę.

Uczenie transferowe wyraźnie pomaga na czystych danych. ResNet18 z wagami ImageNet ma o 0,82 p.p. wyższe macro-F1, a MobileNet aż o 4,10 p.p. Jeszcze lepiej widać to na krzywych uczenia.

Pretrenowany ResNet już po pierwszej epoce miał macro-F1 94,9% (od zera 79,8%) i w trzeciej epoce zbliżył się do 99%, czego wersja od zera nie osiągnęła przez wszystkie 20 epok.

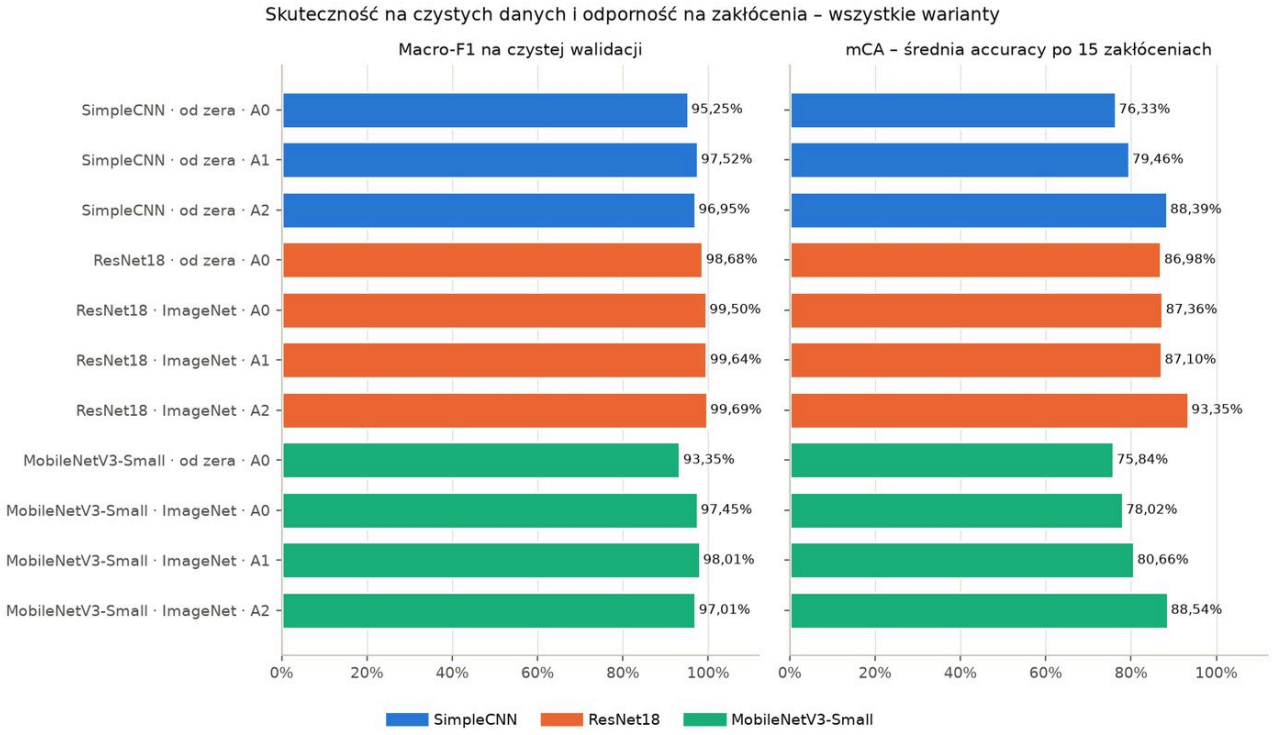
Ciekawie zachował się MobileNet uczony od zera. Przez dwie pierwsze epoki na walidacji praktycznie zgadywał (macro-F1 0,3% i 6,5%), mimo że na treningu miał już ponad 90% accuracy. Dopiero w trzeciej epoce wynik skoczył do 82%. To typowy objaw tego, że na początku treningu statystyki normalizacji wsadowej używane podczas ewaluacji nie nadążają jeszcze za szybko zmieniającymi się wagami. Wersja pretrenowana nie miała tego problemu.

Zaskoczenie: transfer nie poprawia odporności. ResNet18 od zera i pretrenowany mają praktycznie identyczne mCA (86,98% i 87,36%). Na silny szum wersja od zera jest nawet wyraźnie lepsza (91,7% wobec 84,4% na poziomie 3). Wagi z ImageNet pomagają więc rozpoznawać czyste znaki, ale nie przygotowują modelu na zakłócenia.

Wybór do E2. Zgodnie z regułą do E2 przeszły warianty z wyższym macro-F1 w swojej architekturze, czyli oba pretrenowane: ResNet18 i MobileNetV3-Small.

9. E2 i E3: augmentacja i odporność

W E2 wytrenowałem SimpleCNN oraz oba zwycięskie warianty z E1 (pretrenowane ResNet18 i MobileNetV3-Small) z augmentacją A1 i A2. W E3 wszystkie 11 modeli zostało ocenionych na zakłóconej walidacji.

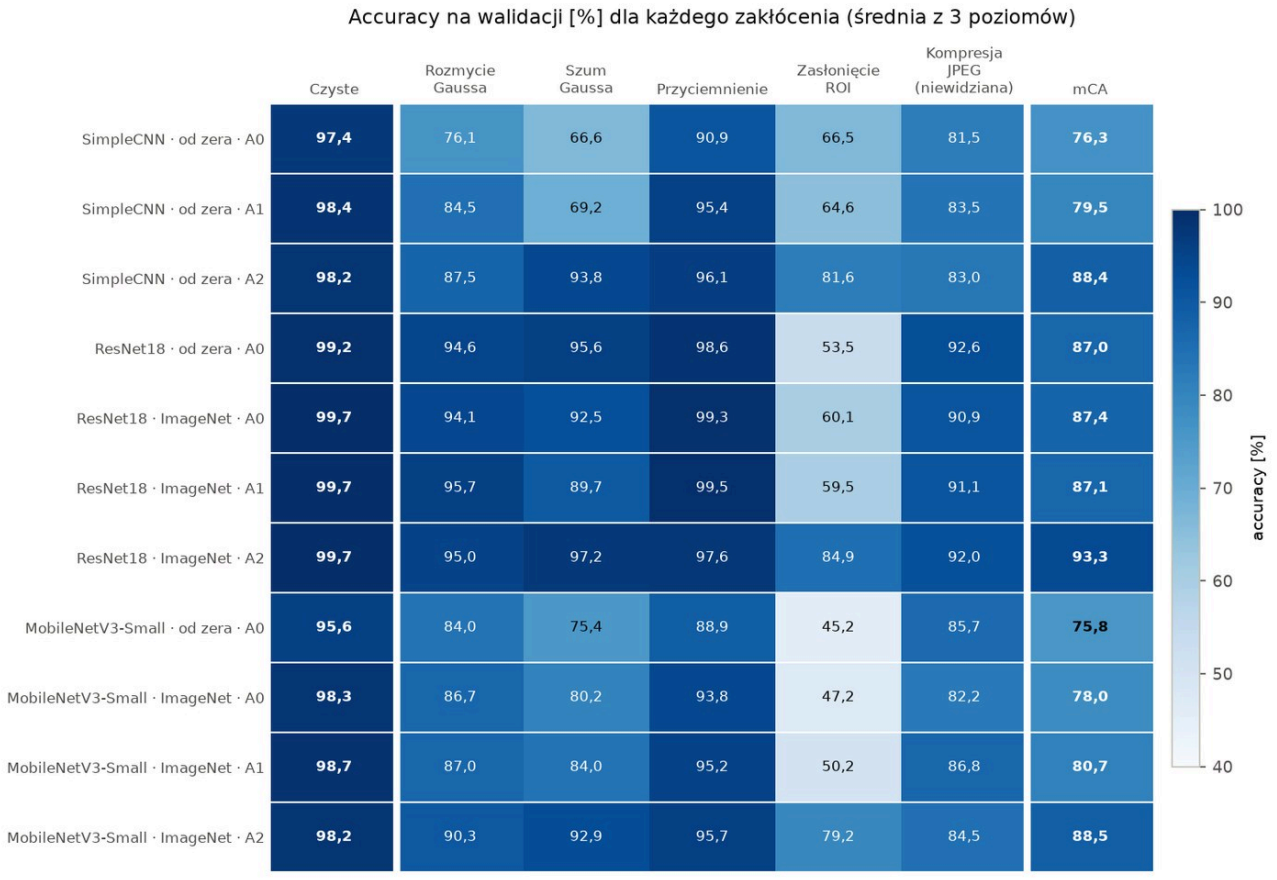


Rys. 10. Macro-F1 na czystej walidacji (lewo) i mCA, czyli średnia accuracy po 15 zakłóceniach (prawo), dla wszystkich 11 wariantów.

Model	Augmentacja	macro-F1	mCA	Względny spadek
SimpleCNN	A0	95,25%	76,33%	21,6%
SimpleCNN	A1	97,52%	79,46%	19,2%
SimpleCNN	A2	96,95%	88,39%	9,9%
ResNet18 (ImageNet)	A0	99,50%	87,36%	12,4%
ResNet18 (ImageNet)	A1	99,64%	87,10%	12,6%
ResNet18 (ImageNet)	A2	**99,69%**	**93,35%**	**6,4%**
MobileNetV3-Small (ImageNet)	A0	97,45%	78,02%	20,6%
MobileNetV3-Small (ImageNet)	A1	98,01%	80,66%	18,3%
MobileNetV3-Small (ImageNet)	A2	97,01%	88,54%	9,9%

9.1. Augmentacja odpornościowa robi ogromną różnicę

A2 podniosła mCA względem A1 we wszystkich trzech architekturach: o 8,9 p.p. w SimpleCNN, o 6,3 p.p. w ResNet18 i o 7,9 p.p. w MobileNet. Względny spadek skuteczności na zakłóceniach zmniejszył się mniej więcej o połowę. Koszt na czystych danych okazał się mały: ResNet18 nawet lekko zyskał (+0,05 p.p.), SimpleCNN stracił 0,57 p.p., a MobileNet równo 1,00 p.p., czyli dokładnie na granicy, którą przyjąłem w hipotezie H2.



Rys. 11. Accuracy na walidacji dla każdego zakłócenia (średnia z trzech poziomów) i dla każdego wariantu.

Największy efekt widać przy zasłonięciu, które bez A2 było dla wszystkich modeli najgroźniejszym zakłóceniem. Pretrenowany ResNet18 z A2 rozpoznaje zasłonięte znaki w 84,9% przypadków zamiast

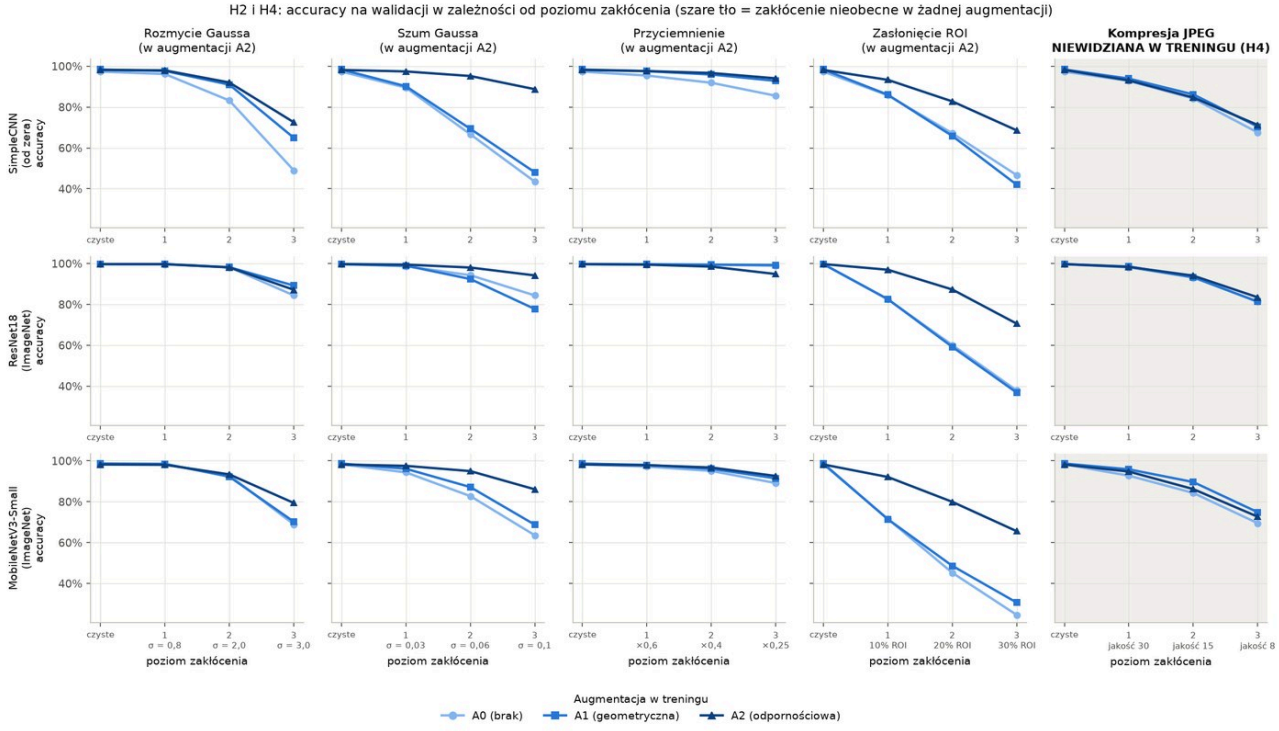
60,1%, a na najsilniejszym poziomie (30% ramki) w 70,6% zamiast 37,9%. Podobnie jest z szumem, gdzie A2 podnosi wynik SimpleCNN z 69% do 94%.

Jedynym zakłóceniem, na którym A2 lekko zaszkodziła, jest przyciemnienie w ResNet18 (97,6% zamiast 99,3%). Bez augmentacji ResNet i tak radził sobie z ciemnymi zdjęciami niemal idealnie, co zgadza się z obserwacją z rozdziału 3.4, że zbiór treningowy jest pełen ciemnych zdjęć.

9.2. Augmentacja geometryczna nie daje odporności

A1 poprawiła wynik na czystych danych (najbardziej w SimpleCNN, o 2,3 p.p.), ale w odporności prawie nic nie zmieniła. Pretrenowany ResNet18 ma z A1 praktycznie to samo mCA co bez augmentacji (87,10% wobec 87,36%), a na silny szum jest nawet słabszy (77,8% wobec 84,4%). Wniosek jest prosty: augmentacja uodparnia model na to, co faktycznie symuluje. Obroty i przesunięcia nie przygotują sieci na rozmycie czy zasłonięcie.

9.3. Zakłócenie niewidziane w treningu (JPEG)



Rys. 12. Accuracy w zależności od poziomu zakłócenia dla A0, A1 i A2. Szare tło oznacza kompresję JPEG, której nie było w żadnej augmentacji.

Na rys. 12 dobrze widać różnicę między zakłóceniami obecnymi w A2 a JPEG. Przy szumie i zasłonięciu ciemna linia A2 wyraźnie odstaje od pozostałych, a przy JPEG wszystkie trzy linie prawie się pokrywają. W liczbach A2 poprawia wynik na JPEG względem A0 tylko o 1,1 do 2,3 p.p., a względem A1 wynik jest niejednoznaczny (ResNet +0,9 p.p., SimpleCNN −0,5 p.p., MobileNet −2,3 p.p.). Odporność nabyta augmentacją praktycznie nie przenosi się więc na nowy rodzaj zakłócenia.

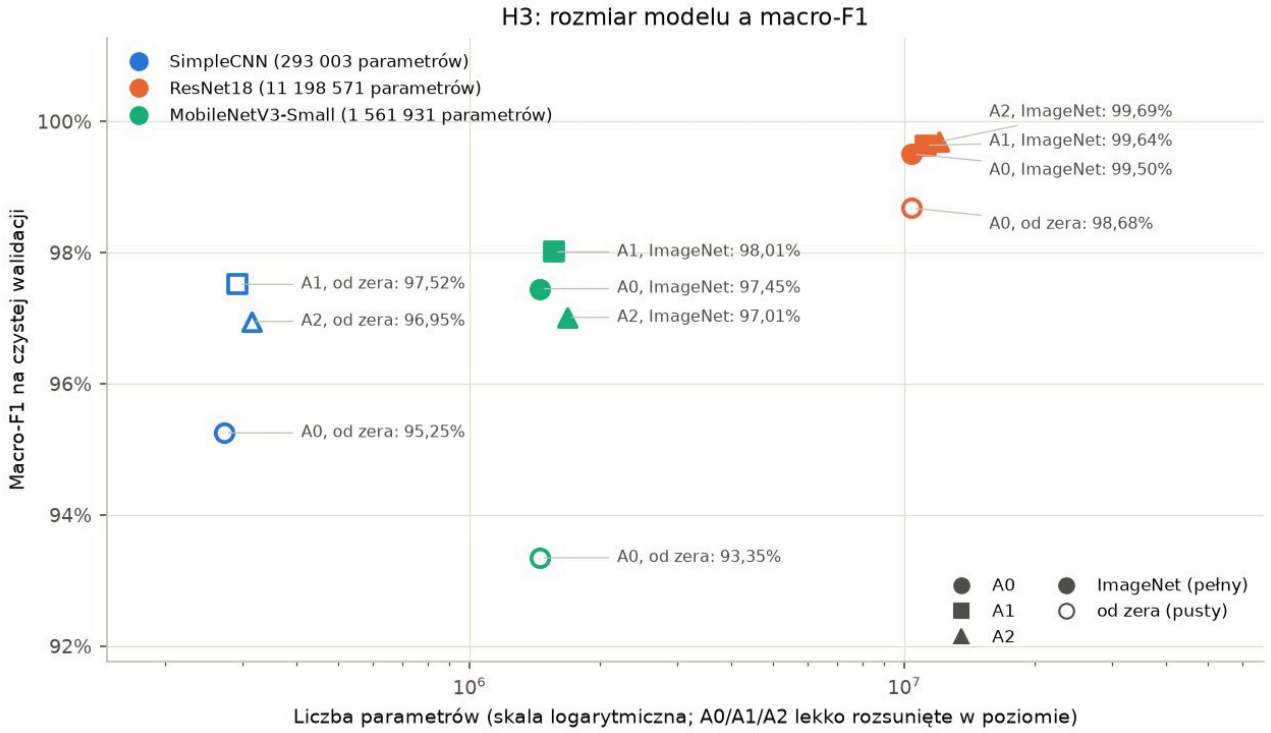
9.4. Augmentacja ważniejsza niż architektura

Najciekawszy wynik całego porównania: SimpleCNN z A2, który ma 293 tysiące parametrów, osiągnął wyższe mCA (88,39%) niż pretrenowany ResNet18 bez augmentacji (87,36%), choć jest od niego 38 razy mniejszy. Jeśli chodzi o odporność, dobór augmentacji okazał się ważniejszy niż wielkość sieci i

wagi z ImageNet. Najlepszy wynik daje jednak dopiero połączenie obu: mocnej, pretrenowanej sieci i augmentacji A2.

10. Rozmiar i szybkość modeli

Hipoteza H3 zakładała, że MobileNetV3-Small osiągnie wynik zbliżony do ResNet18 (różnica macro-F1 poniżej 1 p.p.) przy dużo mniejszym koszcie.



Rys. 13. Macro-F1 na czystej walidacji w zależności od liczby parametrów (skala logarytmiczna). Pełne znaczniki to modele pretrenowane, puste to uczone od zera.

Model	Parametry	Czas predykcji, 4 wątki	Czas predykcji, 1 wątek	Najlepsze macro-F1	Najlepsze mCA
SimpleCNN	0,29 mln	3,4 do 3,5 ms	9,5 do 9,9 ms	97,52%	88,39%
MobileNetV3-Small	1,56 mln	5,4 do 6,2 ms	5,7 do 6,0 ms	98,01%	88,54%
ResNet18	11,20 mln	26,8 do 27,2 ms	88,6 do 91,7 ms	99,69%	93,35%

MobileNet jest rzeczywiście dużo tańszy: ma 7 razy mniej parametrów niż ResNet18 i działa około 5 razy szybciej na 4 wątkach oraz około 15 razy szybciej na jednym. Ciekawostką jest to, że MobileNet prawie nie zyskuje na wielu wątkach, bo jest na tyle lekki, że koszt koordynacji wątków zjada zysk z równoległości.

Na wyniku MobileNet jednak traci. Jego najlepsze macro-F1 (98,01%) jest o 1,68 p.p. niższe od najlepszego ResNeta, a najlepsze mCA o 4,8 p.p. niższe. Przekracza to próg 1 p.p. z hipotezy, więc H3 odrzucam. MobileNet byłby rozsądnym wyborem tam, gdzie liczy się każda milisekunda, ale przy zadaniu, w którym chodzi o bezpieczeństwo, ta różnica jest za duża. Warto też zauważyć, że SimpleCNN z A2 ma prawie taką samą odporność jak MobileNet z A2, przy pięć razy mniejszej liczbie parametrów.

11. Wybór finalnego modelu i stabilność wyniku

11.1. Zastosowanie reguły wyboru

1. **Próg macro-F1.** Najlepsze macro-F1 ma pretrenowany ResNet18 z A2 (99,69%), więc próg wynosi 98,69%. Przechodzą tylko trzy pretrenowane ResNety (A0, A1, A2). ResNet18 uczony od zera odpadł o włos, bo zabrakło mu 0,01 p.p.
2. **Najwyższe mCA.** Spośród trzech pozostałych najwyższe mCA ma wariant z A2 (93,35%), a kolejny (A0) ma 87,36%.
3. **Remis.** Różnica wynosi około 6 p.p., czyli dużo więcej niż próg 0,5 p.p., więc trzeci krok nie był potrzebny.

Zwycięzcą jest ResNet18 z wagami ImageNet i augmentacją A2. Wybór jest jednoznaczny i nie zależy od pomiaru czasu.

11.2. Stabilność (E4)

Zwycięski wariant wytrenowałem jeszcze dwa razy, z innymi ziarnami losowości, żeby sprawdzić, czy jego wynik nie był szczęśliwym trafem.

Seed	macro-F1	mCA	Najlepsza epoka
42	99,69%	93,35%	4
43	99,69%	95,10%	11
44	99,81%	96,43%	15
średnia ± odchylenie	**99,73% ± 0,07**	**94,96% ± 1,55**	

Wynik na czystych danych jest bardzo stabilny, rozrzut to setne części punktu procentowego. Odporność waha się bardziej, ale nawet najsłabszy seed ma ponad 6 p.p. przewagi nad drugim w kolejności wariantem, więc wybór zwycięzcy jest pewny.

Po E2 miałem wątpliwość, bo przebieg z seedem 42 osiągnął najlepszy wynik już w 4. epoce, a wyniki walidacji mocno skakały między epokami. E4 pokazał coś ciekawego: im później trening się zatrzymał, tym wyższa odporność (93,4%, 95,1% i 96,4% dla najlepszej epoki 4, 11 i 15). Early stopping patrzy tylko na czyste dane, więc może przerwać trening, zanim model zdąży się w pełni nauczyć odporności, którą daje augmentacja.

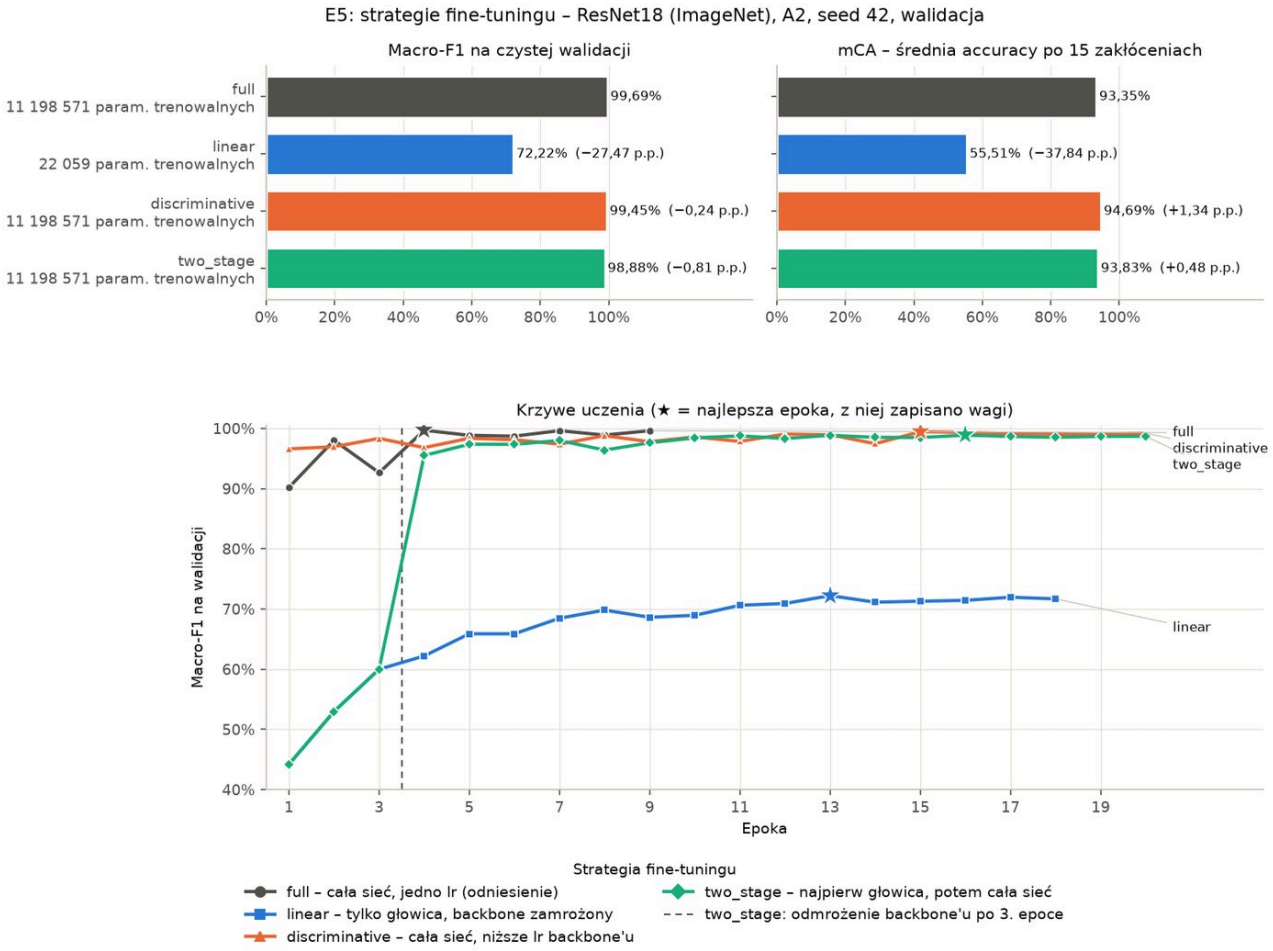
Do oceny końcowej wziąłem model z seedem 42, czyli ten, który wygrał w regule wyboru. Jest najsłabszy z trzech pod względem odporności, więc ocena na teście jest raczej ostrożna niż zawyżona.

12. E5: strategie fine-tuningu

W etapach E1 do E4 modele pretrenowane dostrajałem w najprostszy sposób: podmieniona głowica i trening całej sieci z jednym learning rate 0,001 (rozdział 6.2). Etap E5 dodałem już po ocenie końcowej, żeby sprawdzić, czy sposób dostrajania ma znaczenie i czy wystarczyłoby nauczyć samą ostatnią warstwę. Ponieważ zbiór testowy był już wykorzystany, E5 oceniłem wyłącznie na walidacji, a finalnego modelu nie zmieniałem.

Strategia	Co się uczy	Parametry trenowalne
full (dotychczasowa)	cała sieć, jedno lr = 0,001	11 198 571
linear	tylko głowica, reszta sieci zamrożona	22 059
discriminative [8]	cała sieć, głowica lr = 0,001, pretrenowana część (backbone) lr = 0,0001	11 198 571
two_stage [9]	3 epoki jak linear, potem odmrożenie i dalej jak discriminative	22 059, potem 11 198 571

Wszystkie warianty to ResNet18 z wagami ImageNet, augmentacja A2 i seed 42. Pozostałe ustawienia są takie jak w E1 do E4, a jako wariant *full* posłużył istniejący run z E2. Przy zamrażaniu trzeba było uważać na jedną rzecz: zamrożone warstwy normalizacji wsadowej trzymałem w trybie ewaluacji, bo inaczej ich statystyki zmieniałyby się w trakcie treningu, mimo że same wagi były zamrożone. Sprawdziłem, że w wariancie *linear* wagi i statystyki całego backbone'u po treningu są bitowo identyczne z wagami ImageNet.



Rys. 14. Góra: macro-F1 i mCA na walidacji dla czterech strategii fine-tuningu. Dół: macro-F1 po każdej epoce, gwiazdka oznacza najlepszą epokę, przerywana linia to odmrożenie backbone'u w two_stage.

Strategia	macro-F1	mCA	Względny spadek	Najlepsza epoka	Czas treningu
full	**99,69%**	93,35%	6,4%	4	6,3 min
linear	72,22%	55,51%	29,2%	13	5,8 min
discriminative	99,45%	**94,69%**	**5,1%**	15	10,0 min
two_stage	98,88%	93,83%	5,7%	16	9,2 min

Zakłócenie (średnia z 3 poziomów)	full	linear	discriminative	two_stage
Rozmycie Gaussa	95,0%	57,8%	95,3%	95,0%
Szum Gaussa	97,2%	45,1%	97,8%	97,0%
Przyciemnienie	97,6%	73,4%	99,5%	99,3%
Zasłonięcie	84,9%	45,9%	89,8%	88,3%
Kompresja JPEG (niewidziana)	92,0%	55,4%	91,1%	89,5%

Sama zamiana ostatniej warstwy nie wystarcza. Wariant *linear* osiągnął tylko 72,22% macro-F1, a jego krzywa uczenia wypłaszcza się około 13. epoki, więc dłuższy trening by tego nie zmienił. Cechy z ImageNet były uczone na dużych zdjęciach zwykłych obiektów. U nas obrazy mają 64×64 px, a po zmianie kroku pierwszej konwolucji (rozdział 6.1) wcześniejsze warstwy widzą je w innej skali niż podczas pretreningu. Bez dostrojenia tych warstw cechy po prostu nie pasują do znaków drogowych.

Niższy learning rate dla backbone'u daje najwyższą odporność. Wariant *discriminative* ma mCA wyższe o 1,34 p.p. niż *full*, a macro-F1 niższe tylko o 0,24 p.p. Największe zyski są przy zasłonięciu (+4,9 p.p.) i przyciemnieniu (+1,9 p.p.). Przy przyciemnieniu znika nawet niewielka strata, którą augmentacja A2 powodowała w wariancie *full* (rozdział 9.1). Trening jest też spokojniejszy: już po pierwszej epoce macro-F1 wynosi 96,6% (w *full* 90,2%), a wynik nie zapada się tak jak w *full* w 3. epoce (92,6%). Moim zdaniem powodem jest to, że na początku treningu losowo zainicjalizowana głowica daje duże gradienty. Przy jednym, dużym learning rate te gradienty mocno zmieniają wagi z ImageNet, a mniejszy krok dla backbone'u je chroni.

Trening dwuetapowy nic nie dodał. Pierwsze trzy epoki *two_stage* to w praktyce *linear* (krzywe są niemal identyczne, co było też dobrą kontrolą poprawności kodu). Po odmrożeniu backbone'u wynik od razu skoczył do 95,5%, ale końcowo *two_stage* wypadł nieco gorzej niż *discriminative*.

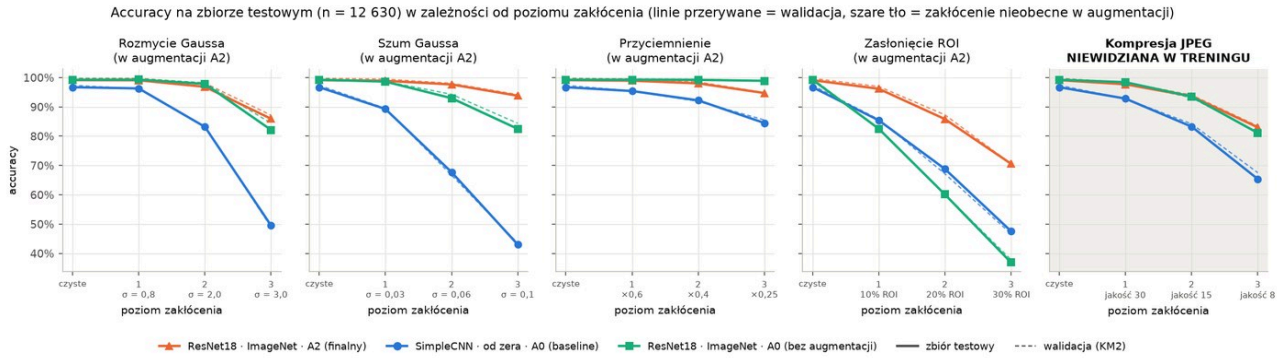
Ostrożnie z wnioskami. E5 to jeden seed, a zysk 1,34 p.p. mCA mieści się w rozrzucie między seedami z E4 (±1,55 p.p.). Do tego *discriminative* zatrzymał się w 15. epoce, a *full* w 4., więc część różnicy może wynikać z dłuższego treningu, który według E4 sprzyja odporności. Gdyby E5 było częścią pierwotnego planu, reguła z rozdziału 5.7 wybrałaby wariant *discriminative* (macro-F1 w granicach 1 p.p. od najlepszego i najwyższe mCA). Na przyszłość przyjąłbym go jako domyślną strategię i potwierdził wynik kilkoma seedami.

13. Ocena końcowa na zbiorze testowym

Zbiór testowy (12 630 zdjęć) użyłem jednorazowo, z tymi samymi metrykami i zakłóceniami co na walidacji. Oceniłem trzy modele: finalny, baseline (SimpleCNN bez augmentacji) oraz pretrenowany ResNet18 bez augmentacji jako punkt odniesienia, żeby sprawdzić, czy efekt augmentacji utrzymuje się na nowych danych. Na podstawie testu nie podejmowałem już żadnych decyzji. Wszystkie progi i wybory zapadły wcześniej, na walidacji, a test służy wyłącznie do raportowania.

Czasy predykcji podaję za pomiarami z rozdziału 7.3, wykonanymi w jednej sesji na bezczynnym komputerze. Model się od tamtej pory nie zmienił, a podczas obliczeń na teście komputer był obciążony innymi programami, więc tamte pomiary nie byłyby wiarygodne.

Model	macro-F1 walidacja	macro-F1 test	Accuracy test	mCA walidacja	mCA test
ResNet18, ImageNet, A2 (finalny)	99,69%	**98,44%**	99,13%	93,35%	**92,72%**
ResNet18, ImageNet, A0 (bez augmentacji)	99,50%	99,00%	99,26%	87,36%	86,89%
SimpleCNN, A0 (baseline)	95,25%	94,64%	96,72%	76,33%	76,30%



Rys. 15. Accuracy na zbiorze testowym w zależności od poziomu zakłócenia dla trzech modeli. Linie przerywane to wyniki z walidacji, szare tło oznacza zakłócenie nieobecne w augmentacji.

13.1. Główny wniosek się potwierdził

Najważniejsze dla mnie było, czy przewaga w odporności nie była artefaktem walidacji. Nie była: na teście finalny model ma mCA o 5,8 p.p. wyższe niż ten sam ResNet18 bez augmentacji, praktycznie tyle samo co na walidacji (6,0 p.p.). Linie przerywane na rys. 15 niemal pokrywają się z ciągłymi, więc zachowanie modeli przy zakłóceniach jest stabilne między zbiorami.

Zakłócenie (średnia z 3 poziomów)	Finalny (A2)	ResNet18 A0	Baseline
Rozmycie Gaussa	94,0%	93,1%	76,3%
Szum Gaussa	96,8%	91,4%	66,7%
Przyciemnienie	97,2%	99,1%	90,7%
Zasłonięcie	84,2%	59,9%	67,3%
Kompresja JPEG (niewidziana)	91,5%	91,0%	80,5%

Największa różnica jest przy zasłonięciu: średnio finalny model rozpoznaje 84,2% zasłoniętych znaków, a ResNet bez augmentacji tylko 59,9%, czyli mniej niż prosty baseline. Na najsilniejszym poziomie (30% ramki) różnica to 70,6% wobec 36,9%. Przy JPEG wszystkie trzy linie na rys. 15 dalej prawie się pokrywają, co potwierdza wniosek z rozdziału 9.3, że odporność nie przenosi się na zakłócenie niewidziane w treningu. Tak jak na walidacji, A2 lekko szkodzi przy przyciemnieniu (97,2% wobec 99,1%).

13.2. Uczciwa niespodzianka na czystych danych

Na czystym teście lepszy okazał się ResNet18 **bez** augmentacji: 99,00% macro-F1 wobec 98,44% dla finalnego modelu. Na walidacji było odwrotnie. Koszt augmentacji A2 na czystych danych wynosi

więc w rzeczywistości około 0,6 p.p., a nie zero, jak wychodziło na walidacji. Nadal mieści się to w progu 1 p.p. z hipotezy H2, ale warto to wyraźnie zaznaczyć.

13.3. Różnica między walidacją a testem

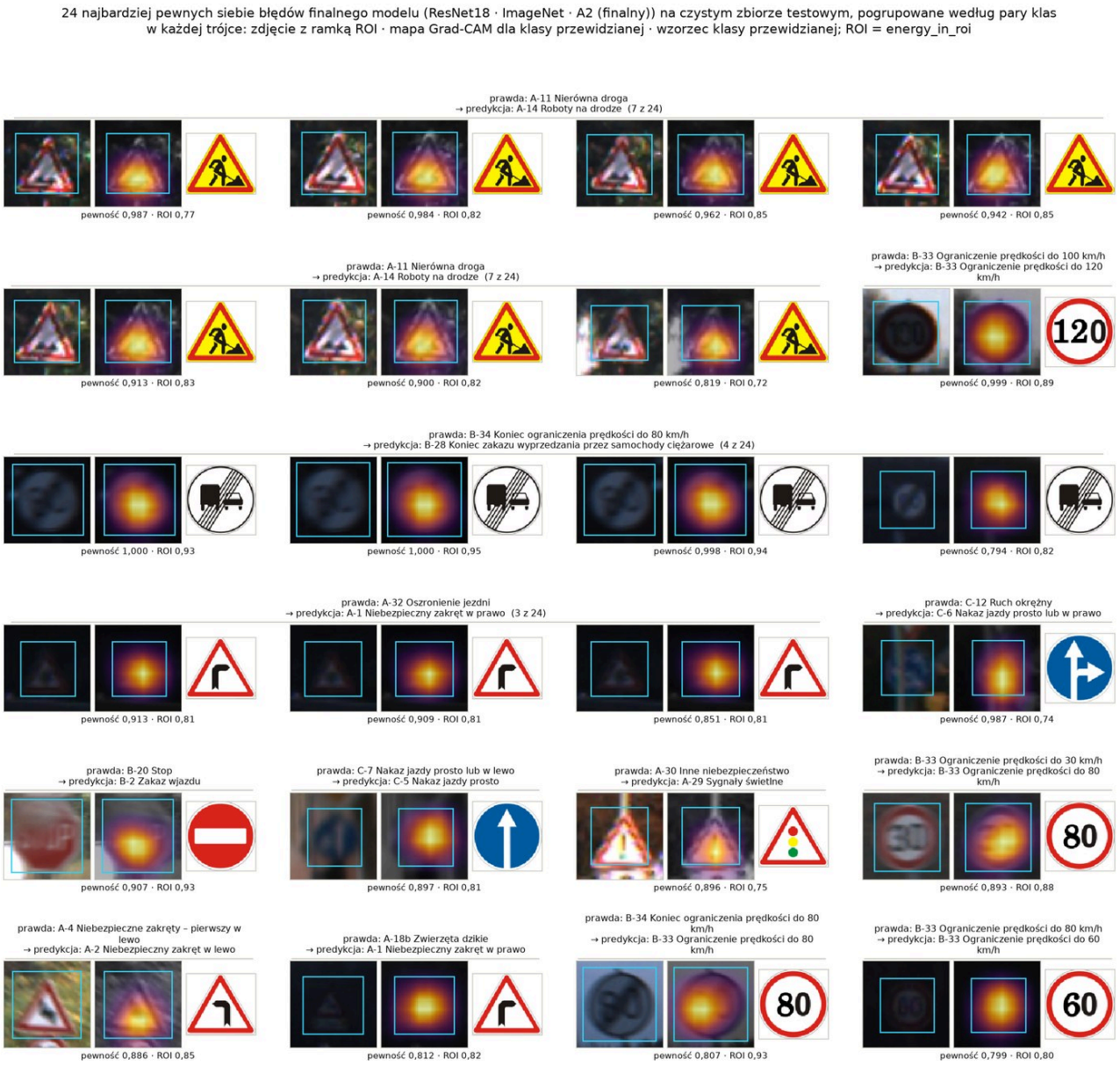
Finalny model stracił na teście najwięcej: 1,25 p.p. macro-F1, podczas gdy pozostałe około 0,5 do 0,6 p.p. To spodziewany efekt wyboru. Spośród 11 wariantów i wielu epok wybrałem ten, któremu na walidacji poszło najlepiej, więc część jego przewagi była szczęściem. W rozdziale 11.2 zwróciłem uwagę, że ten przebieg osiągnął najlepszy wynik już w 4. epoce przy mocno skaczącej walidacji, i test to potwierdza. Odporność (mCA) prawie nie spadła, bo nie ona decydowała o wyborze najlepszej epoki.

14. Analiza błędów

Na czystym teście finalny model pomylił się 110 razy na 12 630 zdjęć. Na etapie analizy danych obstawiałem, że najwięcej pomyłek będzie między ograniczeniami prędkości, które różnią się tylko cyframi. Okazało się, że byłem w błędzie: ograniczenia prędkości są rozpoznawane niemal bezbłędnie, a kłopot sprawiają trójkątne znaki ostrzegawcze.

Prawdziwy znak	Rozpoznany jako	Liczba	Udział w błędach
A-11 Nierówna droga	A-14 Roboty na drodze	19	17%
A-4 Niebezpieczne zakręty, pierwszy w lewo	A-6a Skrzyżowanie z drogą podporządkowaną	14	13%
A-18b Zwierzęta dzikie	A-1 Niebezpieczny zakręt w prawo	9	8%
A-32 Oszronienie jezdni	A-1 Niebezpieczny zakręt w prawo	8	7%
A-30 Inne niebezpieczeństwo	A-16 Przejście dla pieszych	7	6%
B-34 Koniec ograniczenia do 80 km/h	B-33 Ograniczenie do 80 km/h	6	5%

Wszystkie najsłabsze klasy to znaki ostrzegawcze, a najniższe F1 mają „Niebezpieczne zakręty, pierwszy w lewo” (0,89) i „Nierówna droga” (0,90). Wyjaśnienie jest dość proste. Wszystkie te znaki mają identyczny kształt i kolor, a różni je tylko mały piktogram w środku, który przy wejściu 64×64 px ma kilkanaście pikseli. Przy ograniczeniach prędkości cyfry zajmują dużo większą część znaku.



Rys. 16. 24 najbardziej pewne siebie błędy finalnego modelu na czystym teście. W każdej trójce: zdjęcie z ramką ROI, mapa Grad-CAM i wzorzec klasy, którą wskazał model.

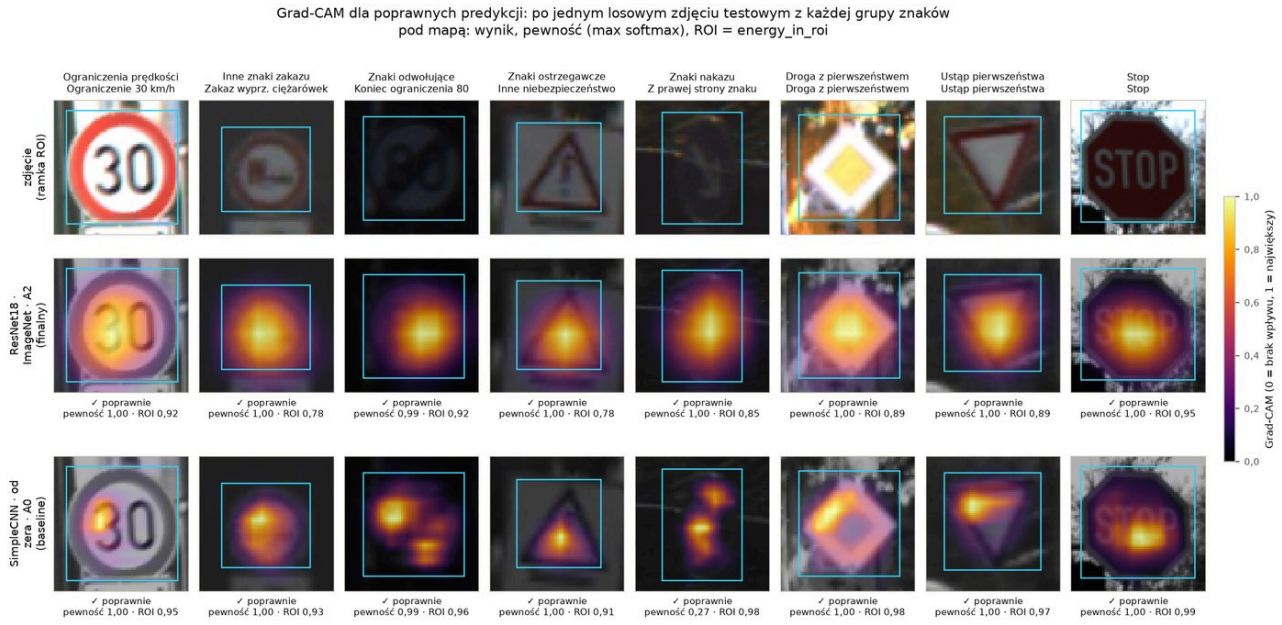
Galeria pokazuje dwie rzeczy, których nie widać w samych liczbach:

- **Błędy są skupione na kilku fizycznych znakach.** Zbiór testowy też składa się z klatek nagrań, więc siedem błędów „nierówna droga → roboty na drodze" to w praktyce kolejne klatki jednego znaku. 110 błędów nie oznacza więc 110 różnych sytuacji.
- **Część błędów dotyczy zdjęć praktycznie nieczytelnych.** Bardzo ciemne znaki „koniec ograniczenia do 80 km/h" model rozpoznał jako „koniec zakazu wyprzedzania przez samochody ciężarowe" z pewnością bliską 100%. To najgroźniejszy typ błędu, bo nie wyłapie go żaden próg pewności (rozdział 16).

15. XAI: na co patrzy model

Do wyjaśniania decyzji użyłem metody Grad-CAM [6]. Tworzy ona mapę ciepła pokazującą, które fragmenty zdjęcia najmocniej wpłynęły na wybraną klasę. Mapę liczę dla ostatniej warstwy

konwolucyjnej: w ResNet18 ma ona rozdzielczość 8×8 (dzięki adaptacji opisanej w rozdziale 6.1, bez której miałaby tylko 2×2), a w SimpleCNN 16×16.



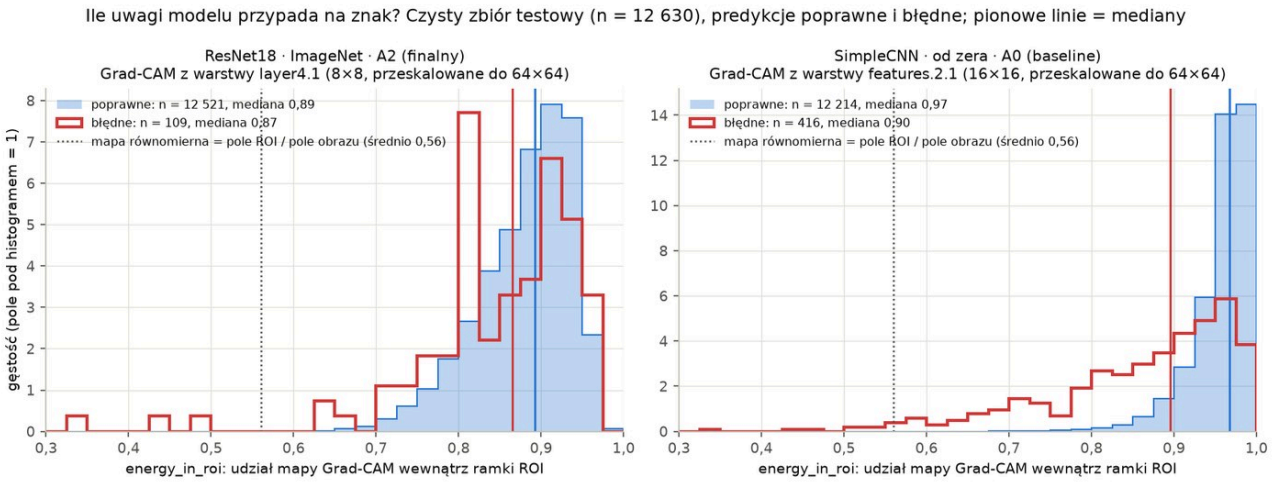
Rys. 17. Grad-CAM dla poprawnych predykcji, po jednym zdjęciu z każdej grupy znaków. Środkowy rząd to model finalny, dolny to baseline.

Na rys. 17 widać różnicę w „stylu patrzenia". Finalny model ma jedną, spójną plamę uwagi na środku znaku, tam gdzie jest piktogram albo liczba. Baseline często skupia się na pojedynczych fragmentach, np. na lewej cyfrze ograniczenia albo na krawędzi znaku „ustąp pierwszeństwa".

15.1. Ilościowo: ile uwagi przypada na znak

Żeby nie opierać się tylko na oglądaniu obrazków, wykorzystałem ramki ROI ze zbioru danych. Dla każdego zdjęcia policzyłem, jaka część mapy Grad-CAM leży wewnątrz ramki ze znakiem. Punktem odniesienia jest pole samej ramki: gdyby mapa była równomierna, wewnątrz ramki leżałoby średnio 56% uwagi.

Model	Predykcje poprawne	Predykcje błędne	Mapa równomierna
ResNet18 A2 (finalny), czysty test	88,1%	84,7%	56%
SimpleCNN (baseline), czysty test	95,8%	86,0%	56%
SimpleCNN, szum poziomu 2	94,2%	71,4%	56%



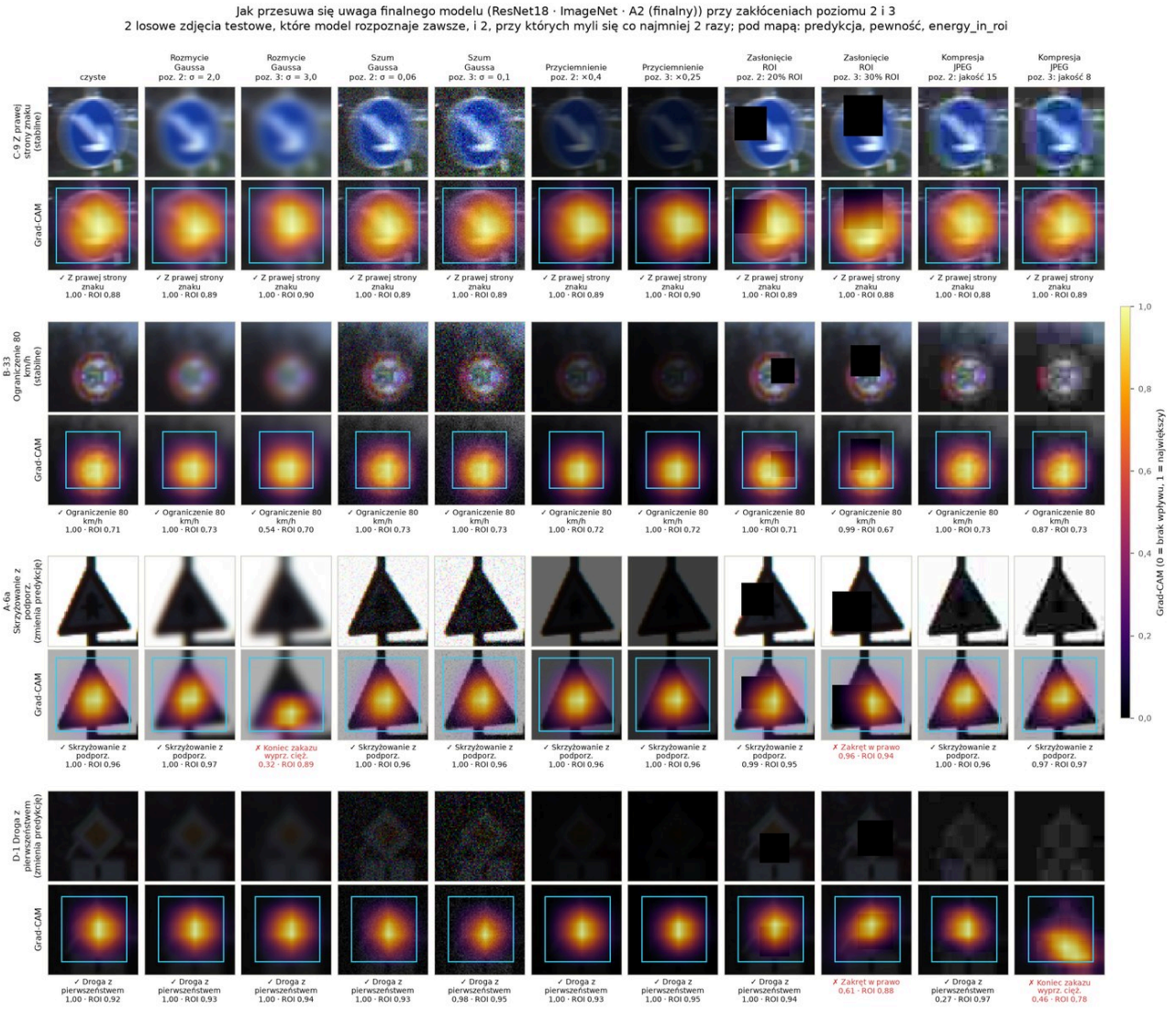
Rys. 18. Rozkład udziału mapy Grad-CAM wewnątrz ramki ROI dla predykcji poprawnych i błędnych na czystym teście.

Oba modele patrzą na znak, a nie na tło: udział uwagi w ramce jest wyraźnie wyższy niż 56%. Ciekawsze jest jednak porównanie trafień i pomyłek w obrębie jednego modelu:

- **Finalny model myli się, patrząc we właściwe miejsce.** Przy błędach udział uwagi w ramce jest prawie taki sam jak przy trafieniach (84,7% wobec 88,1%). Jego pomyłki nie wynikają z tego, że „zgubił" znak, tylko z tego, że nie rozróżnił podobnych piktogramów, co zgadza się z analizą błędów.
- **Baseline przy zakłóceniach gubi znak.** Na zaszumionych zdjęciach udział uwagi w ramce spada przy błędach do 71,4%, czyli model zaczyna reagować na szum w tle.

Dwa zastrzeżenia. Porównywanie wartości bezwzględnych między modelami nie jest do końca uczciwe, bo ostrzejsza mapa 16×16 baseline'u naturalnie mieści się w ramce lepiej niż mapa 8×8 ResNeta. Dlatego wnioski opieram na porównaniu trafień i błędów w obrębie jednego modelu. Po drugie, przy zasłonięciu finalny model kładzie około 22% mapy na czarnym prostokącie. Nie oznacza to, że „patrzy na zasłonięcie": mapa 8×8 to dość gruba plama na środku znaku, a prostokąt też leży na środku.

15.2. Jak uwaga przesuwa się przy zakłóceniach

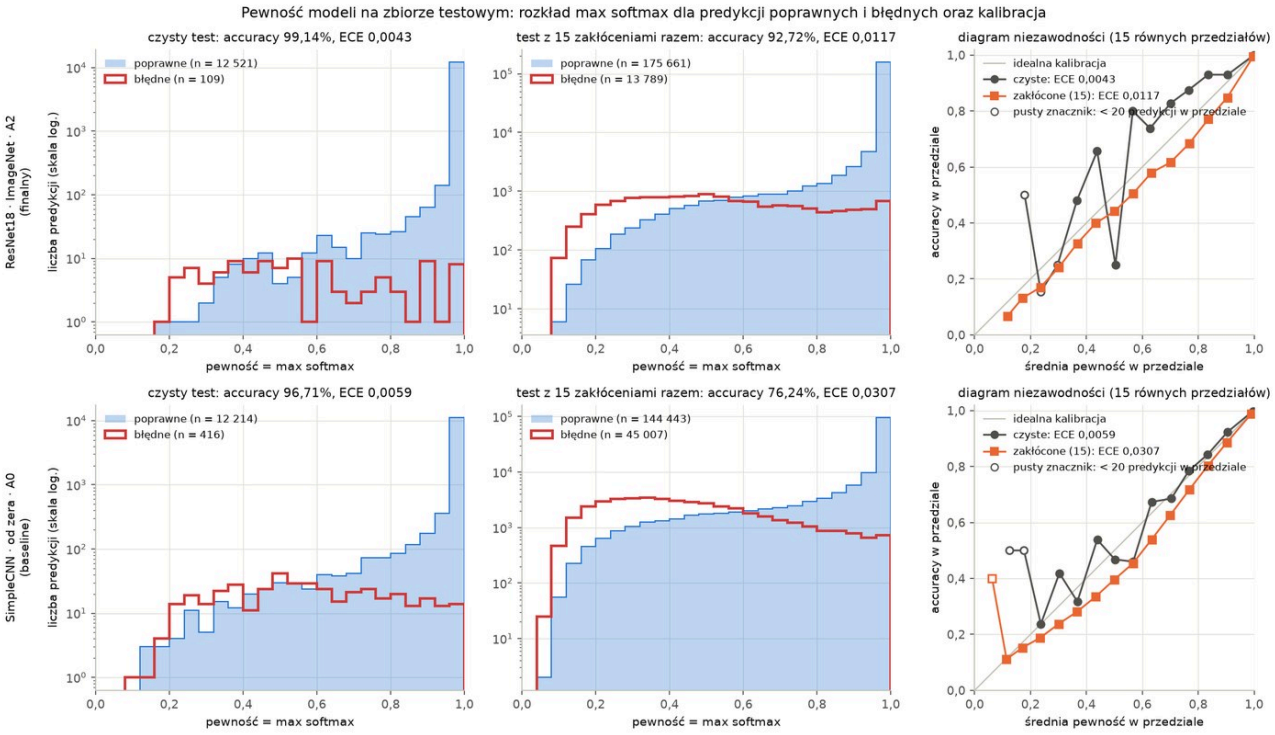


Rys. 19. Grad-CAM finalnego modelu dla czterech zdjęć testowych przy zakłóceniach poziomu 2 i 3. Dwa górne zdjęcia model rozpoznaje zawsze, dwa dolne przy niektórych zakłóceniach zmieniają predykcję.

Dla stabilnych zdjęć mapa wygląda niemal tak samo bez względu na zakłócenie, co dobrze świadczy o odporności. Przy zdjęciach, na których model zmienia zdanie, widać dwa scenariusze. Przy silnym rozmyciu i JPEG mapa ucieka z centrum znaku: na ciemnym znaku „droga z pierwszeństwem" po kompresji JPEG przesuwa się w róg obrazu i model się myli. Przy zasłonięciu mapa zostaje w miejscu, gdzie był piktogram, ale skoro go tam nie ma, model zgaduje na podstawie samego kształtu znaku.

16. Pewność modelu i próg odrzucania

W systemie, który ma wspierać kierowcę, ważne jest nie tylko to, jak często model ma rację, ale też czy „wie, kiedy nie wie". Jako miarę pewności przyjąłem najwyższe prawdopodobieństwo zwracane przez model (max softmax).

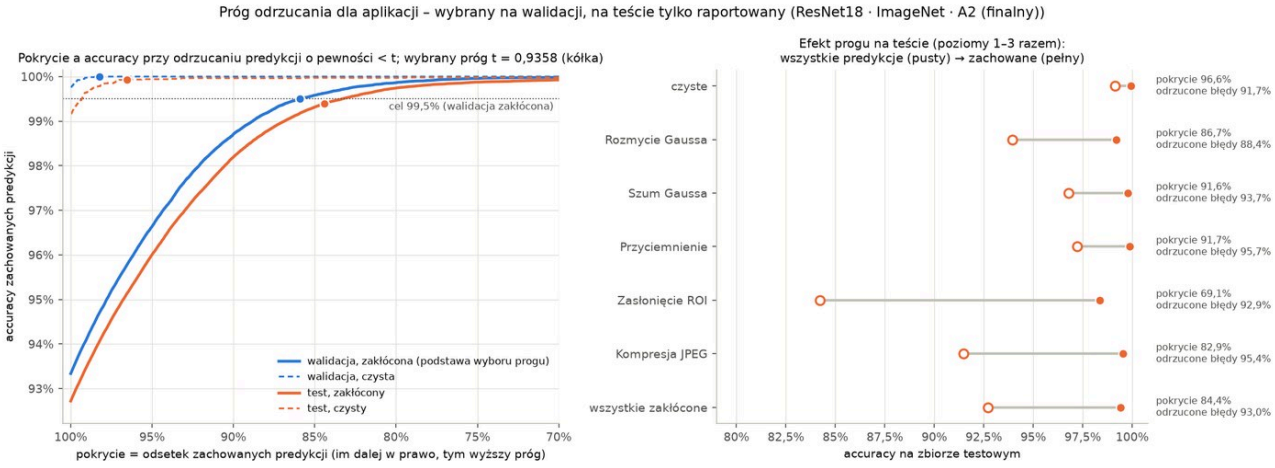


Rys. 20. Rozkład pewności dla predykcji poprawnych i błędnych oraz diagram niezawodności, dla finalnego modelu (góra) i baseline'u (dół).

Finalny model jest dobrze skalibrowany. Średni błąd kalibracji (ECE [7]) wynosi 0,43% na czystym teście i 1,17% na zakłóconym. Przy trafieniach jego średnia pewność to 99,2%, a przy błędach 56,1%, więc pewność rzeczywiście niesie informację o tym, czy wynikowi można ufać. Baseline jest skalibrowany słabiej, szczególnie na zakłóconych danych (ECE 3,07%).

16.1. Próg dla aplikacji

Na tej podstawie wyznaczyłem próg, poniżej którego aplikacja ostrzega, że model nie jest pewny. Próg wybrałem **wyłącznie na walidacji**: to najniższa pewność, przy której zachowane predykcje na zakłóconej walidacji mają co najmniej 99,5% trafień. Wyszło 0,936. Na teście tylko sprawdziłem, jak działa.



Rys. 21. Lewo: accuracy zachowanych predykcji w zależności od odsetka zachowanych predykcji. Prawo: efekt wybranego progu na teście dla każdego zakłócenia.

Zbiór testowy	Zachowane predykcje	Accuracy wszystkich	Accuracy zachowanych	Odrzucone błędy
Czysty	96,6%	99,14%	99,93%	91,7%
Zakłócony (15 wersji razem)	84,4%	92,72%	99,40%	93,0%
Zasłonięcie	69,1%	84,20%	98,40%	92,9%

Próg przeniósł się z walidacji na test bardzo dobrze: na zakłóconych danych zachowane predykcje mają 99,40% trafień przy celu 99,5%. Na czystych zdjęciach model odrzuca tylko 3,4% predykcji, a wyłapuje przy tym ponad 90% swoich błędów. Najwięcej odrzuca przy zasłonięciu (31%), co jest dokładnie tym zachowaniem, jakiego bym chciał: im trudniejsze warunki, tym częściej system przyznaje się do niepewności, zamiast zgadywać.

Słabym punktem są błędy popełnione z bardzo wysoką pewnością. Na czystym teście 3,7% błędów ma pewność powyżej 99%, więc żaden rozsądny próg ich nie zatrzyma.

17. Aplikacja

Całość zamknąłem w aplikacji webowej napisanej w Gradio (`app/app.py`), uruchamianej lokalnie poleceniem `python -m app.app`. Działa na karcie graficznej albo na samym procesorze, gdzie rozpoznanie jednego zdjęcia razem z mapą Grad-CAM trwa około 0,1 s.

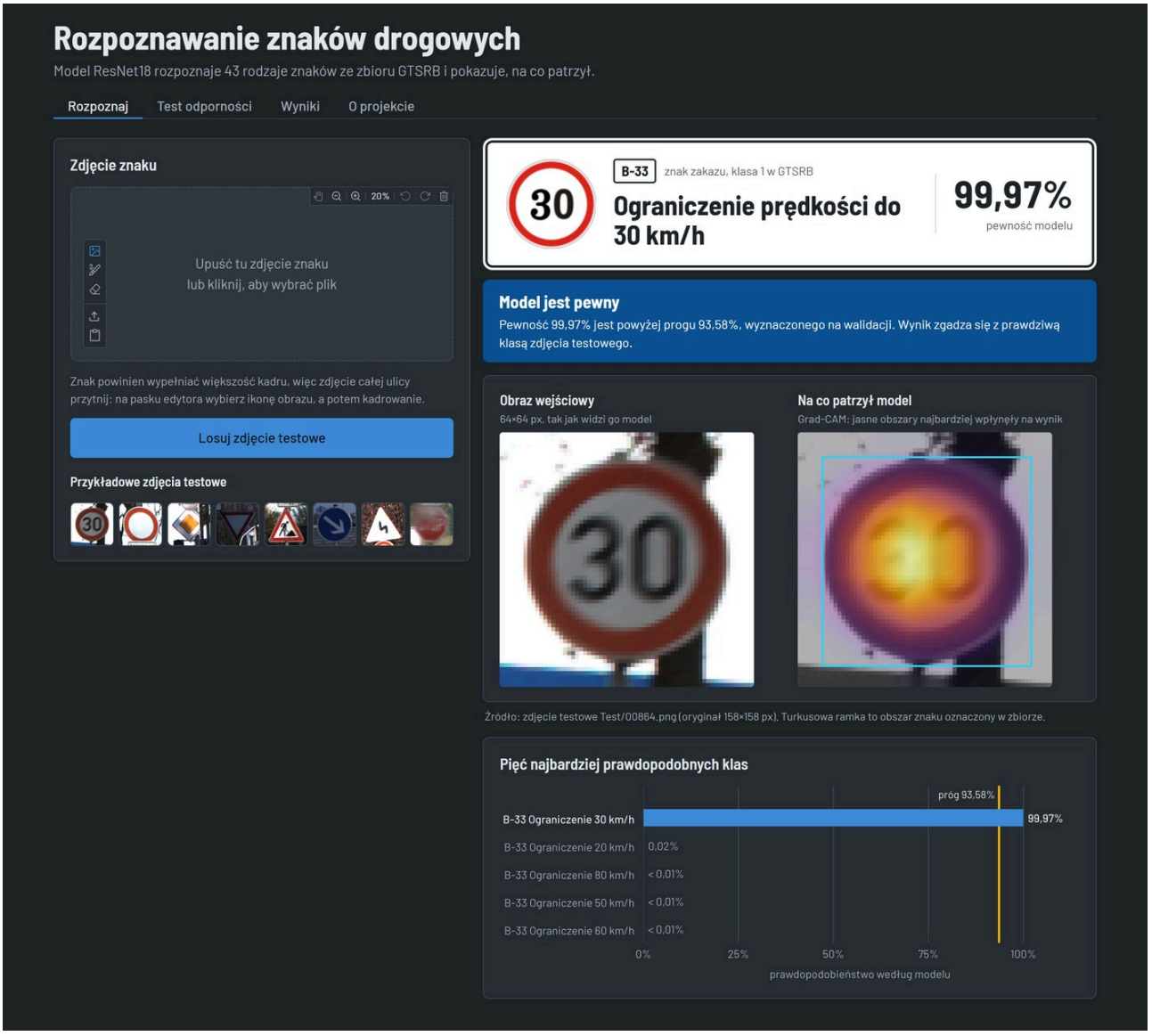
Sprawdziłem, że aplikacja pokazuje dokładnie ten model, który oceniłem: jej predykcje na całym zbiorze testowym zgadzają się z wynikami ewaluacji dla 12 629 z 12 630 zdjęć. Jedyna różnica dotyczy zdjęcia, na którym dwie klasy mają praktycznie równe prawdopodobieństwo (28,9% i 28,5%), więc o wyniku decydują drobne różnice numeryczne. Zakłócone obrazy w aplikacji są identyczne z tymi, na których liczyłem wyniki.

17.1. Wygląd

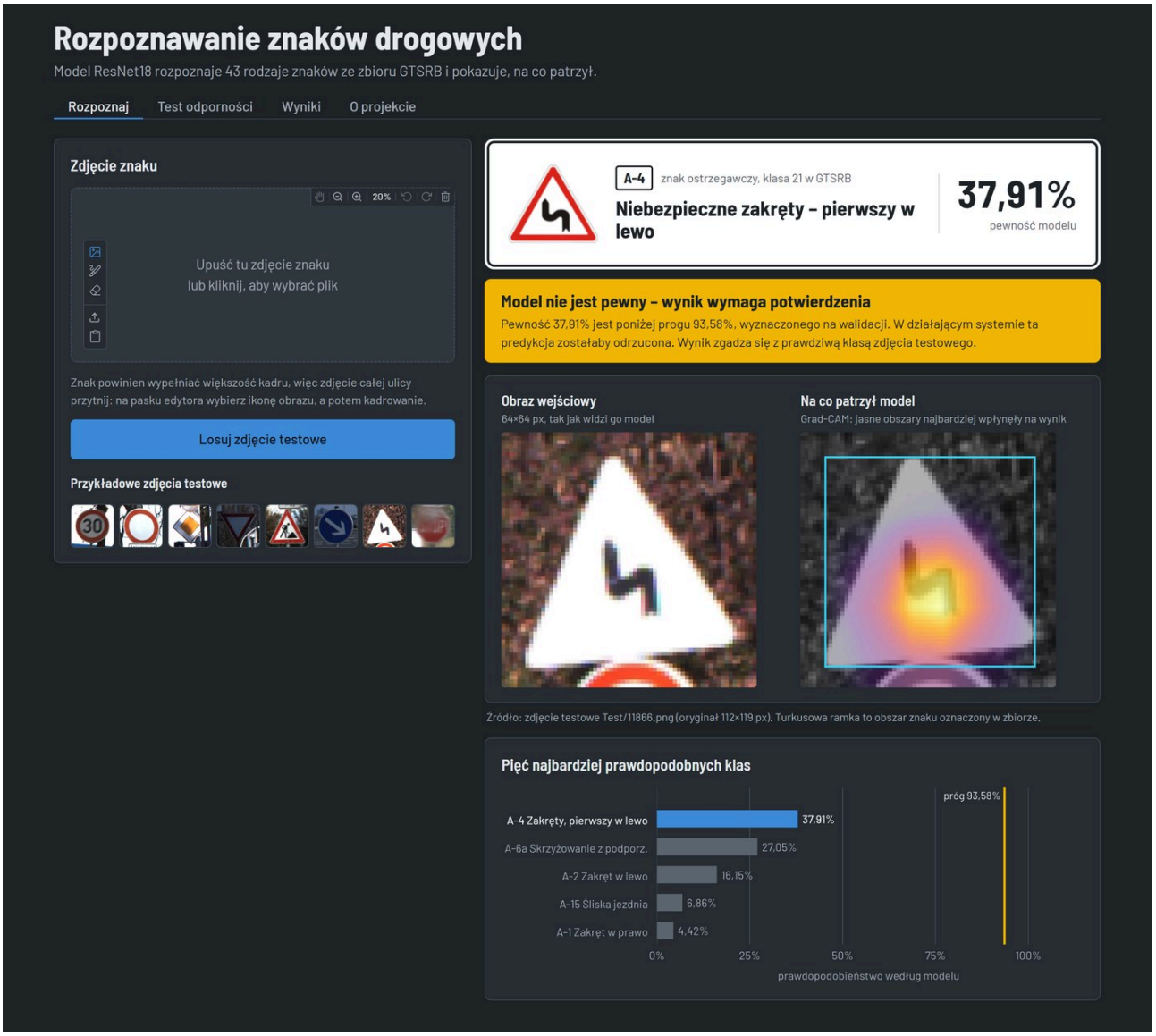
Pierwsza wersja interfejsu wyglądała jak domyślny szablon, więc przeprojektowałem ją pod temat projektu. Interfejs używa kroju Barlow, opartego na DIN 1451, czyli kroju z niemieckich znaków drogowych, z których pochodzi zbiór. Ciemne tło nawiązuje do drogi nocą, a jedynym jasnym elementem jest biała „tabliczka" z wynikiem, jak znak w świetle reflektorów. Kolory pasków stanu pochodzą z oznakowania i mają stałe znaczenie: niebieski to wynik pewny, żółty to niepewność, czerwony to pomyłka.

17.2. Zakładka „Rozpoznaj"

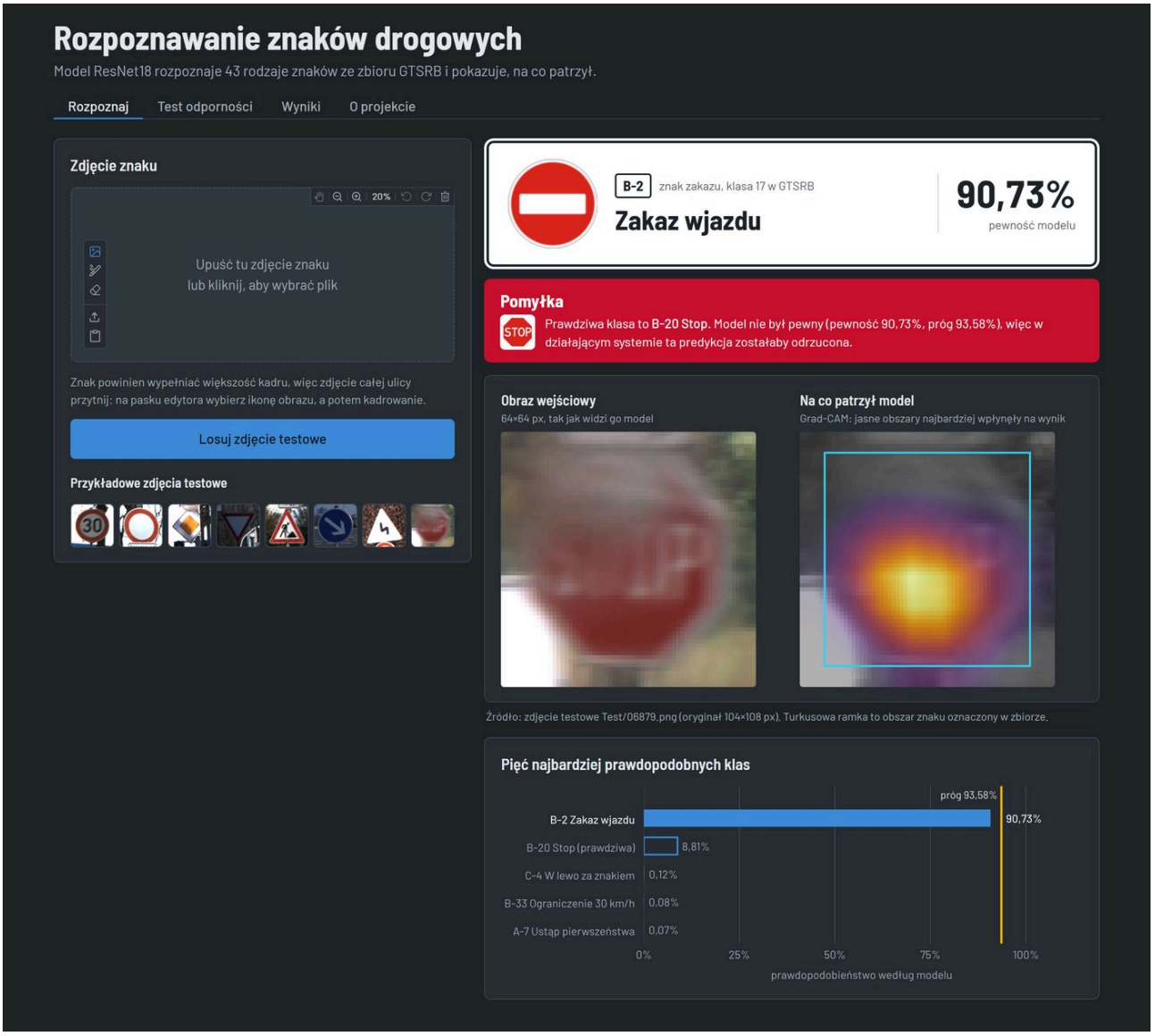
Można wgrać własne zdjęcie i przyciąć je do znaku, wylosować zdjęcie testowe albo kliknąć jedną z ośmiu miniatur. Aplikacja pokazuje rozpoznany znak z polską nazwą i kodem, pewność, mapę Grad-CAM, pięć najbardziej prawdopodobnych klas na tle progu oraz, dla zdjęć testowych, prawdziwą klasę.



Rys. 22. Pewna i poprawna predykcja: ograniczenie prędkości do 30 km/h, pewność 99,97%.



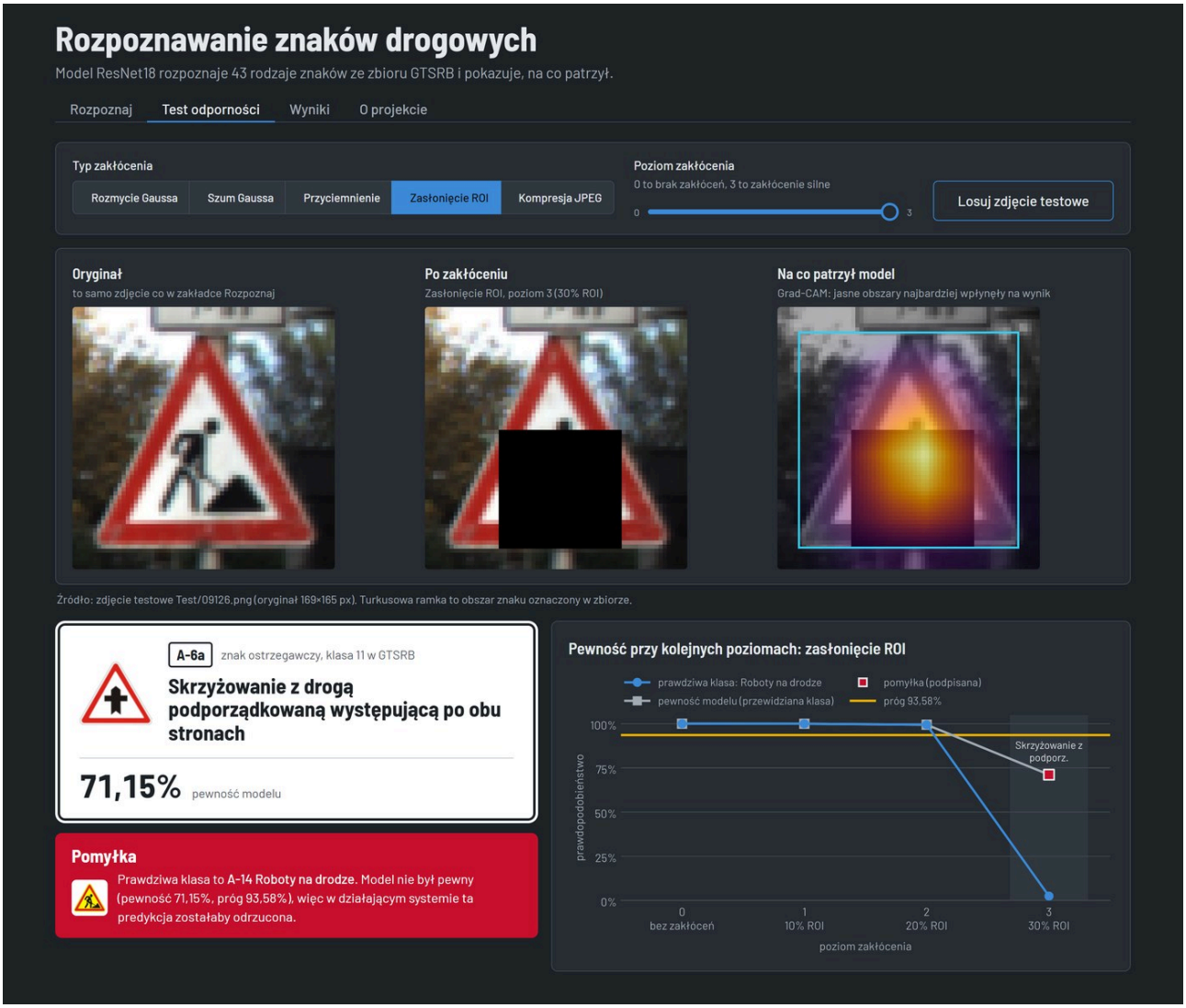
Rys. 23. Wynik poprawny, ale z pewnością 37,9%: żółty pasek informuje, że w działającym systemie ta predykcja zostałaby odrzucona.



Rys. 24. Pomyłka na rozmytym zdjęciu: STOP rozpoznany jako „Zakaz wjazdu". Pewność 90,7% jest poniżej progu, a prawdziwa klasa jest druga na liście.

17.3. Zakładka „Test odporności"

To samo zdjęcie można poddać jednemu z pięciu zakłóceń i suwakiem zmieniać jego siłę. Na żywo widać zakłócony obraz, nową predykcję, mapę uwagi i wykres pewności w funkcji poziomu zakłócenia.

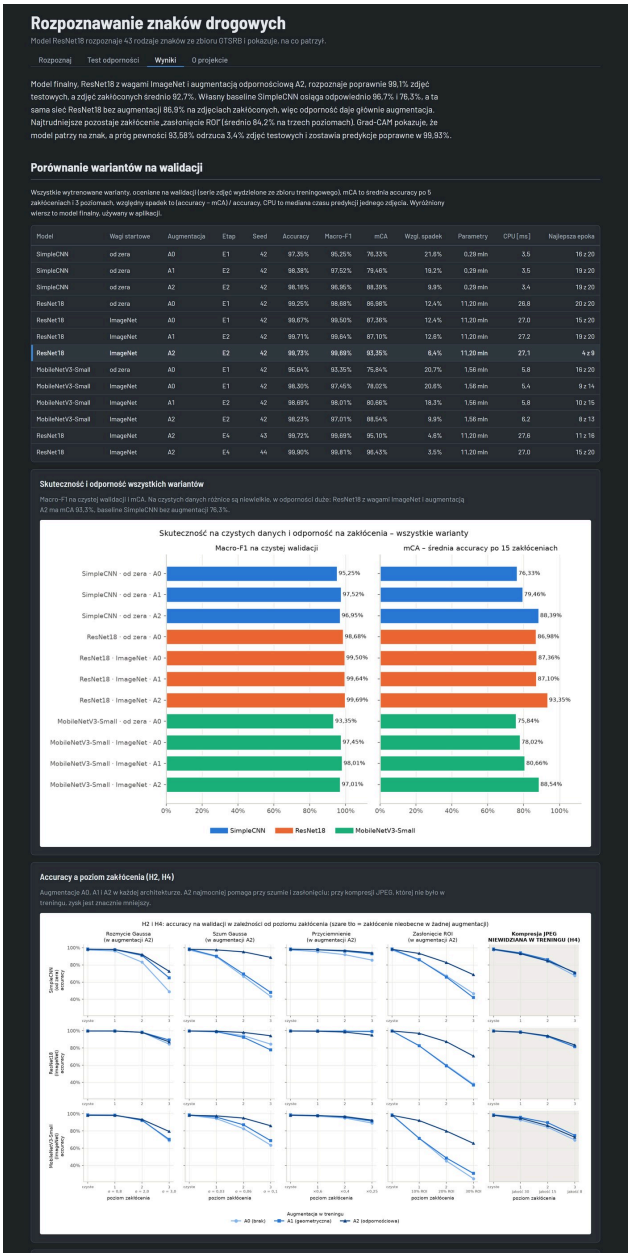


Rys. 25. „Roboty na drodze" przy zasłonięciu 30% ramki: model zmienia odpowiedź na „Skrzyżowanie z drogą podporządkowaną", a pewność prawdziwej klasy spada z niemal 100% do zera.

Ten przykład dobrze pokazuje mechanizm opisany w rozdziale 15.2: mapa uwagi świeci dokładnie tam, gdzie był piktogram robotnika. Model szuka kluczowego fragmentu, a gdy go nie znajduje, wybiera inny znak o tym samym kształcie.

17.4. Zakładki „Wyniki" i „O projekcie"

„Wyniki" zbierają tabele i wykresy z eksperymentów i oceny końcowej, poprzedzone krótkim akapitem wniosków, w którym liczby są czytane bezpośrednio z plików z wynikami. „O projekcie" zawiera krótki opis, link do repozytorium i ograniczenia modelu.



Rys. 26. Zakładka „Wyniki": akapit wniosków, tabela wszystkich wariantów i wykresy porównawcze.

18. Mocne i słabe strony

18.1. Mocne strony

- **Wysoka skuteczność i odporność potwierdzona na niezależnych danych:** 99,1% accuracy na czystym teście i 92,7% średnio na zakłóconym, z przewagą w odporności taką samą jak na walidacji.
- **Model wie, kiedy nie wie.** Jest dobrze skalibrowany, a próg wyznaczony na walidacji wyłapuje około 92% błędów, odrzucając tylko 3,4% czystych zdjęć.
- **Decyzje są wytłumaczalne.** Model patrzy na znak, a jego pomyłki mają sensowne wyjaśnienie: podobne piktogramy w małej rozdzielczości.
- **Rzetelna metodologia.** Podział po seriach bez wycieku, reguły wyboru ustalone przed treningami i zbiór testowy użyty jeden raz.

18.2. Słabe strony i ograniczenia

- **Pewne siebie błędy na bardzo ciemnych zdjęciach.** Część pomyłek ma pewność bliską 100% i żaden próg ich nie zatrzyma.
- **Małe piktogramy w znakach ostrzegawczych.** Przy 64×64 px model myli znaki o tym samym kształcie. Pomogłaby wyższa rozdzielczość wejścia, kosztem dłuższego treningu.
- **Odporność tylko na znane zakłócenia.** Augmentacja nie przygotowała modelu na kompresję JPEG, której nie było w treningu.
- **Klasyfikator, a nie detektor.** Model oczekuje wycinka ze znakiem na środku. Na całym kadrze z tłem rozpoznał STOP jako „zakaz wjazdu" z pewnością 95,5%, a po przycięciu do znaku poprawnie, z pewnością 99,9%. W prawdziwym systemie potrzebny byłby dodatkowo detektor znaków.
- **Zdjęcia, a nie rysunki.** Wzorcowe piktogramy znaków (rysunki na białym tle) model rozpoznaje poprawnie tylko w 30 na 43 przypadkach, np. STOP uznaje za „zakaz wjazdu". To klasyczne przesunięcie dziedziny: model zna tylko zdjęcia z kamery samochodowej.
- **Tylko znaki niemieckie.** Polskie znaki są bardzo podobne, ale nie identyczne (np. żółte tło znaków ostrzegawczych), więc skuteczność na polskich drogach nie jest zweryfikowana.
- **Ograniczenia metody XAI.** Mapa Grad-CAM w rozdzielczości 8×8 jest dość gruba, więc dobrze pokazuje, czy model patrzy na znak, ale nie, na który dokładnie fragment piktogramu.
- **Finalny model nie był najlepszym możliwym.** Wybrana epoka była „szczęśliwa": model stracił na teście 1,25 p.p. macro-F1 względem walidacji, a na czystych danych lepszy okazał się wariant bez augmentacji. E4 i E5 pokazały, że dłuższy trening, kryterium zatrzymania uwzględniające odporność albo niższy learning rate dla backbone'u dałyby prawdopodobnie bardziej odporny model.

19. Podsumowanie

Pytanie badawcze brzmiało: czy uczenie transferowe i odpowiednio dobrana augmentacja ograniczają spadek skuteczności klasyfikatora znaków drogowych na zakłóconych zdjęciach? Po wszystkich etapach odpowiedź jest taka:

- **augmentacja odpornościowa ogranicza ten spadek wyraźnie i trwale:** włączenie jej zmniejszyło średni spadek skuteczności na zakłóceniach mniej więcej o połowę, co potwierdziło się na zbiorze testowym,
- **uczenie transferowe poprawia głównie skuteczność na czystych danych** i przyspiesza trening, ale samo w sobie nie daje odporności,
- **odporność obejmuje głównie zakłócenia, które model widział w treningu,** a na nowe (JPEG) przenosi się w niewielkim stopniu,
- **sposób fine-tuningu ma znaczenie:** sama zamiana ostatniej warstwy daje tylko 72% macro-F1, a niższy learning rate dla pretrenowanej części sieci dał najwyższą odporność z całego projektu (94,69% mCA na walidacji).

Hipoteza	Wynik na walidacji	Weryfikacja na teście
H1 uczenie transferowe	potwierdzona: macro-F1 +0,82 p.p. (ResNet18) i +4,10 p.p. (MobileNet), wyraźnie szybsza zbieżność, bez zysku w odporności	nieweryfikowana, bo modele uczone od zera nie były oceniane na teście
H2 augmentacja odpornościowa	potwierdzona: mCA +6,3 do +8,9 p.p. względem A1, koszt na czystych danych od +0,05 do −1,00 p.p.	potwierdzona: mCA +5,8 p.p., koszt na czystych danych 0,6 p.p.
H3 MobileNet bliski ResNetowi	odrzucona: −1,68 p.p. macro-F1 i −4,8 p.p. mCA, w zamian 7 razy mniej parametrów i około 5 razy szybciej	bez zmian, MobileNet nie był oceniany na teście
H4 odporność na niewidziany JPEG	raczej odrzucona: zysk tylko 1,1 do 2,3 p.p. względem A0	potwierdzona obserwacja: zysk tylko 0,4 p.p. względem A0

Finalny system to pretrenowany ResNet18 z augmentacją A2, który rozpoznaje 99,1% zdjęć testowych i 92,7% zdjęć zakłóconych, sygnalizuje niepewność w przypadkach, w których najczęściej się myli, i pozwala zobaczyć, na co patrzył przy podejmowaniu decyzji.

Największym wnioskiem dla mnie jest to, że w tym projekcie więcej dało przemyślenie danych i sposobu treningu niż wybór architektury. Prosta sieć z dobrą augmentacją okazała się bardziej odporna niż 38 razy większy pretrenowany ResNet bez niej, a decyzje podjęte na etapie analizy danych (podział po seriach, brak odbić lustrzanych, dobór siły augmentacji) okazały się kluczowe dla wiarygodności wyników.

Bibliografia

1. J. Stallkamp, M. Schlipsing, J. Salmen, C. Igel, *The German Traffic Sign Recognition Benchmark: A multi-class classification competition*, IJCNN 2011. PDF
2. GTSRB, German Traffic Sign Recognition Benchmark, kopia zbioru w serwisie Kaggle. kaggle.com
3. K. He, X. Zhang, S. Ren, J. Sun, *Deep Residual Learning for Image Recognition*, CVPR 2016. arXiv:1512.03385
4. A. Howard i in., *Searching for MobileNetV3*, ICCV 2019. arXiv:1905.02244
5. D. Hendrycks, T. Dietterich, *Benchmarking Neural Network Robustness to Common Corruptions and Perturbations*, ICLR 2019. arXiv:1903.12261
6. R. R. Selvaraju i in., *Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization*, ICCV 2017. arXiv:1610.02391
7. C. Guo, G. Pleiss, Y. Sun, K. Q. Weinberger, *On Calibration of Modern Neural Networks*, ICML 2017. arXiv:1706.04599
8. J. Howard, S. Ruder, *Universal Language Model Fine-tuning for Text Classification*, ACL 2018 (fine-tuning z różnym learning rate dla różnych warstw). arXiv:1801.06146
9. A. Kumar, A. Raghunathan, R. Jones, T. Ma, P. Liang, *Fine-Tuning can Distort Pretrained Features and Underperform Out-of-Distribution*, ICLR 2022 (najpierw trening samej głowicy, potem całej sieci). arXiv:2202.10054